# pandas: The Complete Interview Guide for Quant Devs

Oct 19, 2026 · @Naman Jain

## 1. Foundations: what pandas objects really are

pandas puts labels (an Index) on top of typed column arrays. Two ideas explain most of its behaviour: operations align on labels, not positions, and since pandas 3.0 every derived object behaves as a copy (Copy-on-Write). This guide targets pandas 3.0.x, the current major line (3.0.6 as of September 2026).

### 1.1 The three core objects

| Object | What it is | Key attributes |
|---|---|---|
| `Index` | immutable array of labels with a hash table for lookups | `dtype`, `is_unique`, `is_monotonic_increasing`, `name` |
| `Series` | one typed array plus an Index (and a name) | `values` / `array`, `index`, `dtype`, `name` |
| `DataFrame` | dict-like collection of Series sharing one row Index; column labels are also an Index | `columns`, `index`, `dtypes`, `shape` |

```
import numpy as np
import pandas as pd

s = pd.Series([101.5, 102.0, 101.8], index=pd.to_datetime(['2026-10-19',
'2026-10-20', '2026-10-21']), name='close')
df = pd.DataFrame({'close': [101.5, 102.0], 'volume': [1_200, 900], 'sym':
['TCS', 'TCS']})

s.index, s.dtype, s.name
df.dtypes                  # one dtype per column: float64, int64, str
df.shape, df.columns, df.index   # RangeIndex(start=0, stop=2, step=1) by
default
s.to_numpy(); s.array      # NumPy array vs the underlying ExtensionArray
```

## 1.2 Label alignment: the most important behaviour

Arithmetic between pandas objects first aligns them on their labels. Labels missing from either side produce NaN.

```
a = pd.Series([1.0, 2.0, 3.0], index=['INFY', 'TCS', 'HDFC'])
b = pd.Series([10.0, 20.0], index=['TCS', 'INFY'])
a + b            # HDFC -> NaN, INFY 21.0, TCS 12.0 (union of labels, sorted)
a.add(b, fill_value=0)       # HDFC -> 3.0: treat missing as 0
a.values + b.reindex(a.index).values   # explicit positional work after
aligning
```

This is a feature (prices and positions line up by symbol or timestamp automatically) and a trap (two series with slightly different timestamps produce a sea of NaN). When a result has unexpected NaN, check the indexes first.

## 1.3 Inside a DataFrame: blocks and extension arrays

- Columns of the same NumPy dtype are stored together in 2-D blocks managed by the BlockManager. Column-wise operations are fast; iterating rows is slow because each row crosses several blocks.
- Non-NumPy columns (strings, categoricals, nullable integers, Arrow types, tz-aware datetimes) are ExtensionArrays, stored one per column.
- Inserting many columns one at a time fragments the frame and triggers `PerformanceWarning: DataFrame is highly fragmented`. Build new columns in a dict and `pd.concat(axis=1)` once, or use `assign`.
- `df.to_numpy()` on mixed dtypes produces an `object` array. Select numeric columns first.

```
# Slow: 200 separate inserts
for k in range(200):
    df[f'lag{k}'] = df['close'].shift(k)

# Fast: build once
lags = {f'lag{k}': df['close'].shift(k) for k in range(200)}
df = pd.concat([df, pd.DataFrame(lags)], axis=1)
```

## 1.4 Backends: NumPy and Arrow

| Backend | Used for | Notes |
|---|---|---|
| NumPy | default for numbers, bools, datetimes | fastest for numeric math; NaN as missing for floats |

| Backend | Used for | Notes |
|---|---|---|
| Arrow (`ArrowDtype`, `dtype_backend='pyarrow'`) | strings, nested types, nullable everything | fast I/O with Parquet; zero-copy hand-off to Polars, DuckDB |
| `str` dtype (pandas 3.0 default for text) | string columns | backed by PyArrow when installed, else NumPy object |

```
df = pd.read_parquet('ticks.parquet', dtype_backend='pyarrow')   # Arrow-backed columns
df['price'].dtype                                                #
double[pyarrow]
```

## 1.5 Copy-on-Write (CoW)

In pandas 3.0, Copy-on-Write is the default and only mode. Any subset or method result behaves as an independent copy, pandas reuses memory lazily until one side is modified, and `SettingWithCopyWarning` no longer exists.

```
df = pd.DataFrame({'px': [100.0, 101.0, 102.0], 'qty': [5, 10, 15]})

sub = df[df['qty'] > 5]
sub['px'] = 0.0                    # changes sub only; df is untouched

df[df['qty'] > 5]['px'] = 0.0      # chained assignment: never modifies df in pandas 3
df.loc[df['qty'] > 5, 'px'] = 0.0  # correct: one indexing call on the object itself

col = df['px']                     # behaves as a copy, but no data is copied yet
col.iloc[0] = -1                   # the copy happens only now; df keeps its value
```

Practical consequences: drop defensive `.copy()` calls written only to silence warnings, always write through `df.loc[...]` or `df[col] = ...` on the frame you want to change, and remember that NumPy arrays obtained via `to_numpy()` may be read-only views.

### 1.6 The 30-second interview answer

> pandas wraps typed column arrays (NumPy or Arrow) with an Index. Operations are vectorized per column and align on labels, so mismatched indexes produce NaN rather than misaligned data. Internally, same-dtype columns are grouped into blocks, which makes column operations fast and row iteration slow. Since 3.0, Copy-on-Write means every derived object behaves as a copy, chained assignment never works, and you modify data with a single `loc` call on the frame itself.

## 2. dtypes

Each column has one dtype, and pandas offers up to three flavours of most types: NumPy-backed, nullable (masked) and Arrow-backed. The choice decides how missing values behave, how much memory you use, and how fast I/O is.

### 2.1 The dtype map

| Kind | NumPy-backed | Nullable (masked) | Arrow-backed | Missing value |
|---|---|---|---|---|
| Integer | `int64`, `int32`, … | `Int64`, `Int32`, … | `int64[pyarrow]` | none for NumPy ints; `pd.NA` otherwise |
| Float | `float64`, `float32` | `Float64` | `double[pyarrow]` | `NaN` / `pd.NA` |
| Boolean | `bool` | `boolean` | `bool[pyarrow]` | none / `pd.NA` |
| String | `object` (legacy) | `string` | `str` (3.0 default), `string[pyarrow]` | `NaN` for `str`, `pd.NA` for `string` |
| Datetime | `datetime64[s/ms/us/ns]` | | `timestamp[ns][pyarrow]` | `NaT` |
| Datetime with zone | `datetime64[ns, Asia/Kolkata]` | | `timestamp[ns, tz=...][pyarrow]` | `NaT` |

| Kind | NumPy-backed | Nullable (masked) | Arrow-backed | Missing value |
|---|---|---|---|---|
| Duration | `timedelta64[ns]` | | `duration[ns][pyarrow]` | `NaT` |
| Period | `period[D]`, `period[M]` | | | `NaT` |
| Categorical | `category` | | `dictionary<...>[pyarrow]` | `NaN` |
| Other | `interval`, `Sparse[float64, nan]`, `object` | | lists, structs, maps | |

## 2.2 The integer-with-NaN problem

A NumPy `int64` column cannot hold NaN, so one missing value silently converts it to `float64`. Large integers such as exchange order IDs or nanosecond timestamps lose precision above 2**53.

```
s = pd.Series([1, 2, None])
s.dtype                                    # float64: [1.0, 2.0, NaN]
pd.Series([1, 2, None], dtype='Int64')     # [1, 2, <NA>]  stays integer
pd.Series([9_007_199_254_740_993]).astype(float).astype('int64')   #
9007199254740992: off by one

orders = pd.read_csv('orders.csv', dtype={'order_id': 'Int64', 'qty':
'Int64'})
```

## 2.3 Strings in pandas 3.0

```
s = pd.Series(['TCS', 'INFY', None])
s.dtype                                     # str (backed by PyArrow if
installed)
s.isna()                                    # missing sentinel is NaN
s[0] = 5                                    # TypeError: str columns hold only
strings
pd.api.types.is_string_dtype(s)             # the right check; never `s.dtype
== object`
s.astype('string[pyarrow]')                 # the older opt-in string dtype,
uses pd.NA
```

Code written for pandas 2.x that checks `dtype == object` to find text columns breaks on 3.0. Use `select_dtypes(include=['string', 'str'])` or `is_string_dtype`.

## 2.4 Categorical

A categorical stores small integer codes plus a lookup table of categories. It saves memory for repeated labels and speeds up `groupby`, `sort_values` and comparisons.

```
sym = pd.Series(['TCS', 'INFY', 'TCS', 'HDFC'] * 250_000)
sym.memory_usage(deep=True)                  # tens of MB as object/str
cat = sym.astype('category')
cat.memory_usage(deep=True)                  # about 1 MB: int8 codes + 3
categories
cat.cat.codes, cat.cat.categories

rating = pd.Categorical(['B', 'A', 'C'], categories=['C', 'B', 'A'],
ordered=True)
rating < 'A'                                 # ordered comparisons follow the
category order

cat.cat.add_categories(['SBIN'])
cat.cat.remove_unused_categories()
```

Pitfalls: assigning a value not in the categories raises; concatenating categoricals with different categories falls back to `object` / `str` (use `union_categoricals`); grouping by a categorical with `observed=False` produces empty groups for every unused category, so pass `observed=True`.

## 2.5 Datetimes and resolution

pandas supports `s`, `ms`, `us` and `ns` resolutions. In pandas 3.0, parsing strings defaults to microseconds, falling back to nanoseconds only when the strings carry nanosecond digits.

```
pd.to_datetime(['2026-10-19 09:15:00']).dtype             # datetime64[us]
in pandas 3
pd.to_datetime(['2026-10-19 09:15:00.123456789']).dtype   # datetime64[ns]
pd.to_datetime([1_760_845_500], unit='s').dtype           # datetime64[s]

ts = pd.Series(pd.to_datetime(['2026-10-19 09:15:00']))
ts.astype('int64')                       # microseconds since epoch, NOT
nanoseconds, on 3.0!
ts.dt.as_unit('ns').astype('int64')      # force the unit before converting
to integers

ist = ts.dt.tz_localize('Asia/Kolkata')  # datetime64[us, Asia/Kolkata]
ist.dt.tz_convert('UTC')
```

Code that turns timestamps into epoch integers with `astype('int64')` silently produces values 1000 times smaller after upgrading. Always call `.dt.as_unit('ns')` first, or keep feeds as `int64` nanoseconds from the start.

## 2.6 Converting types

```
df = df.astype({'qty': 'int64', 'px': 'float64', 'sym': 'category'})
pd.to_numeric(s, errors='coerce')                # bad strings -> NaN
pd.to_numeric(s, downcast='float')               # smallest float that fits
(float32)
pd.to_datetime(s, format='%Y-%m-%d %H:%M:%S', errors='coerce', utc=True)
df.convert_dtypes()                              # best nullable dtypes
df.convert_dtypes(dtype_backend='pyarrow')       # Arrow-backed equivalents
df.select_dtypes(include='number')               # numeric columns only
df.select_dtypes(include=['datetime', 'datetimetz'])
```

Always pass an explicit `format=` to `to_datetime` for large files: format inference per value is slow, and ambiguous dates (03/04/2026) can parse inconsistently.

## 2.7 Measuring and reducing memory

```
df.info(memory_usage='deep')
df.memory_usage(deep=True).sort_values(ascending=False)

def shrink(df):
    out = df.copy()
    for c in out.select_dtypes('float').columns:
        out[c] = pd.to_numeric(out[c], downcast='float')
    for c in out.select_dtypes('integer').columns:
        out[c] = pd.to_numeric(out[c], downcast='integer')
    for c in out.select_dtypes(include=['str', 'string', 'object']).columns:
        if out[c].nunique() < 0.5 * len(out):
            out[c] = out[c].astype('category')
    return out
```

Downcast with care: `float32` keeps about 7 significant digits (fine for features, not for PnL), and `int32` overflows on cumulative volume or notional in paise.

## 2.8 dtypes for market data

| Column | dtype | Reason |
|---|---|---|
| Exchange timestamp | `datetime64[ns]` (UTC) or `int64` ns | nanosecond ordering; float loses precision |

| Column | dtype | Reason |
|---|---|---|
| Price | `float64`, or `int64` ticks | exact comparison and grouping by level |
| Quantity, volume, OI | `int64` or `Int64` | overflow safety; nullable when gaps exist |
| Symbol, exchange, side | `category` | repeated labels, fast groupby |
| Order ID | `int64` / `Int64` or `str` | never float |
| Flags | `bool` or `boolean` | |
| Features for ML | `float32` | memory and bandwidth |

# 3. Creating, inspecting and selecting data

Use `loc` for labels, `iloc` for positions, `at` / `iat` for single values, and one indexing call per assignment. Ambiguous `[]` indexing is the source of most selection bugs.

## 3.1 Constructors

```
pd.DataFrame({'sym': ['TCS', 'INFY'], 'px': [3900.5, 1500.2]})          #
dict of columns
pd.DataFrame([{'sym': 'TCS', 'px': 3900.5}, {'sym': 'INFY', 'px': 1500.2}])
# list of records
pd.DataFrame(np.random.rand(3, 2), columns=['a', 'b'], index=['x', 'y', 'z'])
pd.DataFrame.from_records(rows, columns=['ts', 'px', 'qty'])            #
tuples or structured array
pd.DataFrame.from_dict({'TCS': {'px': 3900}, 'INFY': {'px': 1500}},
orient='index')
pd.DataFrame({'a': s1, 'b': s2})                                         #
Series are ALIGNED on index

idx = pd.date_range('2026-10-01', periods=5, freq='B')                  #
business days
pd.Series(range(5), index=idx)
```

## 3.2 Inspecting

```
df.head(), df.tail(3), df.sample(5, random_state=0)
df.shape, df.dtypes, df.columns, df.index
df.info(memory_usage='deep')
df.describe(percentiles=[0.01, 0.5, 0.99])          # numeric summary
df.describe(include='all')
df['sym'].value_counts(normalize=True, dropna=False)
df.nunique()
df.isna().sum()                                     # missing values per
column
df.duplicated(subset=['ts', 'sym']).sum()
df.index.is_unique, df.index.is_monotonic_increasing
```

## 3.3 Selection rules

| Syntax | Selects by | Slices include the end? | Notes |
|---|---|---|---|
| `df['px']` | column label | | returns a Series |
| `df[['px', 'qty']]` | list of column labels | | returns a DataFrame |
| `df[2:5]` | row positions | no | row slicing with `[]`: avoid, use `iloc` |
| `df[mask]` | boolean rows | | mask must align with the index |
| `df.loc[r, c]` | labels or boolean masks | **yes** | the default choice |
| `df.iloc[r, c]` | integer positions | no | NumPy-style |
| `df.at[r, c]` / `df.iat[i, j]` | one label / one position | | fastest scalar access |
| `df.loc[lambda d: d['px'] > 100]` | callable | | handy inside method chains |

```
df = pd.DataFrame({'px': [100., 101., 102., 103.], 'qty': [5, 10, 15, 20]},
                  index=['a', 'b', 'c', 'd'])
df.loc['b':'c']                     # rows b AND c: label slices include the
end
df.iloc[1:3]                        # rows 1 and 2: positions exclude the end
df.loc['b', 'px']                   # scalar
df.loc[['a', 'd'], ['qty']]
df.iloc[-1]                         # last row as a Series
df.iloc[:, 0]                       # first column
df.loc[df['qty'] > 8, 'px']         # boolean rows + column

s = pd.Series([10, 20, 30], index=[2, 1, 0])
s[0]                                # 30 in pandas 3: integer keys are ALWAYS
labels on a Series
s.iloc[0]                           # 10: positional
```

## 3.4 Filtering

```
df[(df['px'] > 100) & (df['qty'] < 20)]        # & | ~ with parentheses,
never and/or
df[df['sym'].isin(['TCS', 'INFY'])]
df[df['px'].between(100, 102, inclusive='both')]
df[df['sym'].str.startswith('HDFC')]
df[~df['sym'].isin(blacklist)]

limit = 100
df.query('px > @limit and qty < 20')           # @ refers to Python variables
df.query('`bid size` > 0')                     # backticks for column names
with spaces

df['px'].where(df['qty'] > 5)                  # keep shape: NaN where the
condition is False
df['px'].mask(df['qty'] > 5, 0.0)              # replace where the condition
is True
```

## 3.5 Setting values safely

```
df.loc[df['qty'] > 10, 'px'] = 0.0           # one call: correct under Copy-
on-Write
df['notional'] = df['px'] * df['qty']
df['side'] = np.where(df['qty'] > 0, 'B', 'S')
df = df.assign(ret=lambda d: d['px'].pct_change(),
               logret=lambda d: np.log(d['px']).diff())
df = df.assign(notional=pd.col('px') * pd.col('qty'))   # pandas 3.0
expression syntax

df.loc['e'] = [104.0, 25]                     # setting a missing label
appends a row (slow in loops)
df.insert(0, 'sym', 'TCS')                    # insert a column at a
position
df = df.rename(columns={'px': 'price'})
df = df.drop(columns=['notional'])
qty = df.pop('qty')                           # remove and return a column
```

Never write `df[cond]['col'] = x` or `df['col'][cond] = x`: under Copy-on-Write they modify a temporary and leave `df` unchanged.

## 3.6 Sorting and ranking

```
df.sort_values(['sym', 'ts'], ascending=[True, True], kind='stable')
df.sort_values('px', ascending=False, na_position='last')
df.sort_index()
df.nlargest(5, 'volume'); df.nsmallest(5, 'spread')
df['px'].rank(method='average', pct=True)       # ties: average, min, max,
first, dense
```

Sort ticks with `kind='stable'` so rows with identical timestamps keep their arrival order.

## 3.7 Duplicates

```
df.duplicated(subset=['ts', 'sym', 'px', 'qty'], keep='first')
df.drop_duplicates(subset=['ts', 'sym'], keep='last')   # e.g. keep the
latest quote per timestamp
df[df.index.duplicated(keep=False)]                     # all rows with a
repeated index label
```

### 3.8 Iterating (and why not to)

| Approach | Speed | Comment |
|---|---|---|
| vectorized column ops | fastest | always try first |
| `df.to_numpy()` + NumPy or Numba | fast | for stateful loops |
| `itertuples()` | slow | namedtuples, keeps dtypes per field |
| `apply(axis=1)` | slower | builds a Series per row |
| `iterrows()` | slowest | a Series per row; mixed dtypes upcast to object |

When a loop is unavoidable (an order-book replay, a path-dependent payoff), drop to NumPy arrays and compile with Numba rather than iterating DataFrame rows.

## 4. The Index

The Index drives alignment, lookups, slicing, joins and resampling. Keep it unique and sorted, and choose deliberately between a long (date, symbol) MultiIndex and a wide date-by-symbol matrix.

### 4.1 Index types

| Type | Typical use |
|---|---|
| `RangeIndex` | default 0..n-1, stores only start/stop/step |
| `Index` (int, float, str) | symbols, IDs, strikes |
| `DatetimeIndex` | time series; supports partial-string slicing, `resample`, time-based `rolling` |
| `TimedeltaIndex` | time since open, latency buckets |
| `PeriodIndex` | calendar periods (months, quarters) |
| `CategoricalIndex` | small fixed label sets |
| `IntervalIndex` | bins, for example price or volatility buckets |
| `MultiIndex` | hierarchical keys such as (date, symbol) or (expiry, strike, type) |

## 4.2 Setting and resetting

```
df = df.set_index('ts')                          # column -> index
df = df.set_index(['date', 'sym'])               # MultiIndex
df = df.set_index('sym', append=True)            # add a level
df = df.reset_index()                            # index -> column(s)
df = df.reset_index(drop=True)                   # discard the index
df = df.rename_axis('timestamp')                 # name the index
df.index.name, df.index.names
```

## 4.3 Reindexing and aligning

```
full = pd.date_range('2026-10-19 09:15', '2026-10-19 15:30', freq='1min')
bars = bars.reindex(full)                            # missing minutes -> NaN
rows
bars = bars.reindex(full, method='ffill')            # carry the last bar
forward (index must be sorted)
quotes.reindex(trade_times, method='ffill', tolerance=pd.Timedelta('1s'))   #
as-of lookup with staleness limit

a, b = s1.align(s2, join='inner')                    # keep only common labels
prices.reindex(columns=universe)                     # fix the column set and
order

idx1.union(idx2); idx1.intersection(idx2); idx1.difference(idx2)
```

## 4.4 Lookup behaviour and performance

- A unique index uses a hash table; a sorted (monotonic) index allows binary search and fast slicing. Check `df.index.is_unique` and `is_monotonic_increasing`.
- With a non-unique index, `df.loc[label]` returns a DataFrame instead of a Series, which breaks code expecting one row.
- Slicing a non-sorted DatetimeIndex with bounds that are not present raises `KeyError`. Call `sort_index()` once after loading.
- `get_indexer(targets)` returns positions for many labels at once (-1 if missing), useful for vectorized mapping.

```
idx = pd.Index(['HDFC', 'INFY', 'TCS'])
idx.get_loc('INFY')                          # 1
idx.get_indexer(['TCS', 'SBIN', 'HDFC'])     # [ 2 -1  0]
```

## 4.5 Partial-string indexing on a DatetimeIndex

```
ts = ts.sort_index()
ts.loc['2026-10']                                    # the whole month
ts.loc['2026-10-19']                                 # the whole day
ts.loc['2026-10-19 09:15':'2026-10-19 09:30']        # inclusive time range
ts.loc[pd.Timestamp('2026-10-19 09:15'):]            # pass Timestamps, not
datetime.date objects
```

## 4.6 MultiIndex

```
dates = pd.date_range('2026-10-19', periods=3, freq='B')
syms = ['HDFC', 'INFY', 'TCS']
mi = pd.MultiIndex.from_product([dates, syms], names=['date', 'sym'])
panel = pd.DataFrame({'close': np.random.rand(9)}, index=mi).sort_index()

panel.loc[(pd.Timestamp('2026-10-19'), 'TCS')]      # one row
panel.loc[pd.Timestamp('2026-10-19')]               # all symbols on a date
panel.xs('TCS', level='sym')                        # one symbol across dates
idx = pd.IndexSlice
panel.loc[idx[:, ['INFY', 'TCS']], :]               # all dates, two symbols
panel.loc[idx['2026-10-20':, :], 'close']           # date range, every
symbol

panel.index.get_level_values('sym')
panel.swaplevel('date', 'sym').sort_index()
panel.droplevel('sym')
panel.groupby(level='sym')['close'].mean()
wide = panel['close'].unstack('sym')                # date x sym matrix
```

Rules: sort a MultiIndex (`sort_index()`) before slicing it, or you get `UnsortedIndexError` or a slow path; slicing inside a tuple needs `pd.IndexSlice` or `slice(None)`; after filtering, `index.remove_unused_levels()` drops stale level values.

## 4.7 Long vs wide panels

| Layout | Shape | Best for |
|---|---|---|
| Long (tidy) | one row per (date, symbol), MultiIndex or columns | storage, joins with reference data, groupby per symbol, Parquet files |
| Wide | dates as rows, symbols as columns | vectorized cross-sectional math (ranks, z-scores, `corr`, portfolio returns via `@`) |

Convert with `unstack('sym')` / `stack()` or `pivot` / `melt` (section 9). Typical workflow: store long, compute signals wide, join back long.

# 5. Missing data

pandas has three missing-value markers (NaN, `pd.NA`, NaT), skips them in reductions by default, and offers fills that are easy to misuse. In financial data the main risk is not the gap itself but filling it with information from the future or from another instrument.

## 5.1 The three markers

| Marker | Appears in | Behaviour |
|---|---|---|
| `np.nan` | `float64`, `str` (pandas 3) | `nan == nan` is False; propagates through arithmetic |
| `pd.NA` | nullable (`Int64`, `boolean`, `string`) and Arrow dtypes | three-valued logic; `pd.NA == 1` is `NA`; `bool(pd.NA)` raises |
| `pd.NaT` | datetime, timedelta, period | the time equivalent of NaN |

```
pd.NA | True, pd.NA & False      # True, False: Kleene logic, the result is
known
pd.NA | False                    # <NA>: unknown
if s.isna().any(): ...           # never `if value == np.nan` or `if pd.NA:`
```

## 5.2 Detecting

```
df.isna().sum()                      # missing per column
df.isna().mean().sort_values()       # fraction missing
df[df.isna().any(axis=1)]            # rows with any gap
df['px'].notna()
df['px'].first_valid_index(), df['px'].last_valid_index()
```

## 5.3 Reductions and `min_count`

```
s = pd.Series([np.nan, np.nan])
s.sum()                 # 0.0: an all-missing sum is 0 by default
s.sum(min_count=1)      # nan: require at least one valid value
s.mean(), s.count()     # nan, 0
df.mean(skipna=False)   # propagate NaN instead of skipping
```

A daily PnL of 0 because every fill was missing is not the same as a flat day. Use `min_count=1` in resample and groupby sums of PnL or volume.

## 5.4 Filling

```
df.fillna(0)
df.fillna({'qty': 0, 'px': df['px'].median()})          # per-column values
df['px'].ffill()                                         # last known value
df['px'].ffill(limit=5)                                  # at most 5
consecutive fills
df['px'].ffill(limit_area='inside')                      # only between valid
values
df['px'].bfill()                                         # next value: LOOK-
AHEAD in a backtest
df['px'].interpolate(method='time')                      # uses both
neighbours: look-ahead

long_df['px'] = long_df.groupby('sym')['px'].ffill()    # fill within each
symbol only
long_df['px'] = long_df.groupby(['sym', 'date'])['px'].ffill()   # never
across the overnight gap

df['px'] = df['px'].replace({0.0: np.nan, -1.0: np.nan}) # feed sentinels ->
missing first
```

## 5.5 Dropping

```
df.dropna()                                 # any missing value in the row
df.dropna(subset=['px', 'qty'])             # only these columns matter
df.dropna(how='all')                        # only fully empty rows
df.dropna(thresh=3)                         # keep rows with at least 3 valid
values
df.dropna(axis=1, how='all')                # drop empty columns
```

## 5.6 Look-ahead and leakage traps

- `bfill` and `interpolate` use values that were not yet known at that timestamp.
- `fillna(df['x'].mean())` uses the full-sample mean, including the future. Use an expanding or rolling mean.
- A forward fill on a long (date, symbol) table without `groupby('sym')` copies the previous symbol's last value into the next symbol's first row.
- A forward fill across the overnight gap treats yesterday's close as today's 09:15 price. Group by session date.

- `dropna()` on a forward-return target removes the last k rows per symbol, which is correct; doing it before computing features can silently shift alignment.

### 5.7 Stale prices distort statistics

Forward-filled illiquid prices produce runs of zero returns, which understate volatility and pull correlations toward zero. Track staleness explicitly:

```
stale = df['px'].isna()
df['px'] = df['px'].ffill()
df['bars_since_trade'] = stale.groupby((~stale).cumsum()).cumsum()
clean = df[df['bars_since_trade'] <= 3]       # exclude prices older than 3
bars
```

### 5.8 Missing keys in groupby and merge

```
df.groupby('sector')['ret'].mean()                  # rows with NaN sector
are DROPPED by default
df.groupby('sector', dropna=False)['ret'].mean()    # keep them as their own
group

pd.merge(a, b, on='isin')                           # NaN keys MATCH each
other in pandas (unlike SQL)
```

Drop or fill missing join keys before merging, or rows with unknown identifiers will be cross-joined with each other.

## 6. Transforming columns

Prefer built-in vectorized methods and accessors ( `.str` , `.dt` , `.cat` ), then `np.where` / `np.select` , then `map` with a dict; reach for `apply` last. Method chains with `assign` and `pipe` keep transformations readable and free of intermediate variables.

## 6.1 Vectorized arithmetic and series methods

```
df['notional'] = df['px'] * df['qty']
df['ret'] = df['px'].pct_change(fill_method=None)   # no hidden forward fill
df['logret'] = np.log(df['px']).diff()
df['px_prev'] = df['px'].shift(1)
df['cum_vol'] = df['qty'].cumsum()
df['run_max'] = df['px'].cummax()
df['clipped'] = df['ret'].clip(-0.05, 0.05)
a.add(b, fill_value=0); a.sub(b); a.mul(b); a.div(b)  # flex methods with
fill_value and axis
wide.sub(wide.mean(axis=1), axis=0)                   # subtract the row mean
from each row
wide.div(wide.sum(axis=1), axis=0)                    # row-normalize
```

The `axis=0` argument on flex methods aligns a Series with the row index; plain `wide - row_mean` would align it with the columns and fill everything with NaN.

## 6.2 Method chaining with `assign` and `pipe`

```
def add_session(d, open_='09:15', close='15:30'):
    t = d.index.time
    return d.assign(in_session=(t >= pd.Timestamp(open_).time()) & (t <=
pd.Timestamp(close).time()))

out = (
    raw
    .rename(columns=str.lower)
    .astype({'px': 'float64', 'qty': 'int64'})
    .set_index('ts').sort_index()
    .pipe(add_session)
    .query('in_session')
    .assign(notional=lambda d: d['px'] * d['qty'],
            ret=lambda d: d['px'].pct_change())
)
```

Inside `assign`, use a `lambda d:` (or `pd.col` in pandas 3.0) so the expression sees the frame at that point of the chain, including columns created earlier in the same chain.

## 6.3 `map`, `apply`, `transform`, `agg`

| Method | Input to your function | Output | Speed |
|---|---|---|---|
| `Series.map(dict or Series)` | lookup table | same length; missing keys -> NaN | fast (hash join) |
| `Series.map(func)` | each element | same length | slow (Python per element) |
| `DataFrame.map(func)` | each element | same shape | slow (replaced `applymap`) |
| `Series.apply(func)` / `DataFrame.apply(func)` | each element / each column | anything | slow-ish, flexible |
| `DataFrame.apply(func, axis=1)` | each row as a Series | anything | slowest |
| `transform(func)` | column or group | same shape as input | depends on func |
| `agg(['mean', 'std'])` | column or group | reduced | fast for built-in names |

```
sector = {'TCS': 'IT', 'INFY': 'IT', 'HDFCBANK': 'BANK'}
df['sector'] = df['sym'].map(sector)                    # unmapped symbols ->
NaN: check afterwards
lot_size = ref.set_index('sym')['lot']
df['lot'] = df['sym'].map(lot_size)                     # map with a Series =
vectorized lookup

df[['px', 'qty']].agg(['mean', 'std', 'min', 'max'])
df.agg({'px': 'last', 'qty': 'sum'})
```

## 6.4 Conditional columns

```
df['side'] = np.where(df['qty'] > 0, 'BUY', 'SELL')

conds = [df['ret'] > 0.01, df['ret'] < -0.01]
df['signal'] = np.select(conds, [1, -1], default=0)

df['bucket'] = df['ret'].case_when([            # pandas 2.2+: first matching
condition wins
    (df['ret'] > 0.02, 'big_up'),
    (df['ret'] < -0.02, 'big_down'),
])                                               # rows matching nothing keep
their original value
```

## 6.5 The `.str` accessor

```
s = pd.Series(['NIFTY26OCT24500CE', 'BANKNIFTY26OCT52000PE', 'RELIANCE'])
s.str.len(); s.str.upper(); s.str.strip(); s.str.zfill(8)
s.str.contains('NIFTY', regex=False)            # literal match is faster
than regex
s.str.startswith(('NIFTY', 'BANKNIFTY'))
s.str.slice(0, 5); s.str[-2:]
s.str.replace('NIFTY', 'NIF', regex=False)

pattern = r'^(?P<und>[A-Z]+?)(?P<yy>\d{2})(?P<mon>[A-Z]{3})(?P<strike>\d+)(?
P<type>CE|PE)$'
parts = s.str.extract(pattern)                  # DataFrame of named groups;
non-matches -> NaN
parts['strike'] = pd.to_numeric(parts['strike'])

pd.Series(['a,b', 'c,d']).str.split(',', expand=True)   # split into columns
```

## 6.6 The `.dt` accessor

```
t = df['ts']                               # datetime64 column
t.dt.date; t.dt.time; t.dt.hour; t.dt.minute; t.dt.dayofweek; t.dt.day_name()
t.dt.floor('5min'); t.dt.ceil('1h'); t.dt.round('1s')    # bucket timestamps
t.dt.normalize()                           # midnight of each day, keeps dtype
t.dt.tz_localize('Asia/Kolkata').dt.tz_convert('UTC')
t.dt.strftime('%Y%m%d')
t.dt.is_month_end
(t - t.dt.normalize()).dt.total_seconds()   # seconds since midnight
df['latency_us'] = (df['recv_ts'] - df['exch_ts']).dt.total_seconds() * 1e6
```

### 6.7 Binning with `cut` and `qcut`

```
pd.cut(df['ret'], bins=[-np.inf, -0.02, 0, 0.02, np.inf], labels=['big_dn',
'dn', 'up', 'big_up'])
pd.cut(df['spread_bps'], bins=5)                    # 5 equal-width bins
pd.qcut(df['mcap'], q=10, labels=False)             # deciles 0..9 (equal
counts)
pd.qcut(df['x'], q=5, duplicates='drop')            # ties can collapse edges

# Cross-sectional deciles per day (long panel)
df['decile'] = df.groupby('date')['mom'].transform(lambda x: pd.qcut(x, 10,
labels=False, duplicates='drop'))
```

`cut` bins are right-closed `(a, b]` by default; pass `right=False` for `[a, b)` and `include_lowest=True` to include the very first edge.

### 6.8 Ranking

```
df['px'].rank()                                  # average rank for ties
df['px'].rank(method='first')                    # ties broken by order of
appearance
df['px'].rank(method='dense', ascending=False)
df.groupby('date')['mom'].rank(pct=True)         # cross-sectional percentile
rank per day
wide.rank(axis=1, pct=True)                      # same on a date x symbol
matrix
```

### 6.9 Replacing values and winsorizing

```
df['exch'] = df['exch'].replace({'NSE_EQ': 'NSE', 'BSE_EQ': 'BSE'})
df['px'] = df['px'].replace(0, np.nan)
lo, hi = df['ret'].quantile([0.01, 0.99])
df['ret_w'] = df['ret'].clip(lo, hi)                      # full-sample:
fine for EDA, look-ahead in a backtest
q = df.groupby('date')['ret'].transform
df['ret_w'] = df['ret'].clip(q(lambda x: x.quantile(0.01)), q(lambda x:
x.quantile(0.99)))   # per day
```

## 7. GroupBy

GroupBy is split, apply, combine. Know which of the four verbs you need: `agg` reduces each group to one row, `transform` returns a result aligned to the original rows, `filter` keeps or drops whole groups, and `apply` is the slow general fallback.

## 7.1 Grouping keys

```
g = df.groupby('sym')                                    # one column
g = df.groupby(['date', 'sym'])                          # several columns
g = df.groupby(df['ts'].dt.date)                         # a derived Series
g = df.groupby(level='sym')                              # an index level
g = df.groupby(pd.Grouper(key='ts', freq='5min'))      # time buckets on a
column
g = df.groupby([pd.Grouper(key='ts', freq='D'), 'sym'])
g = df.groupby('sym', sort=False, observed=True, dropna=False)
```

`sort=False` skips sorting the group keys (faster); `observed=True` avoids empty groups for unused categories; `dropna=False` keeps NaN keys as a group.

## 7.2 Aggregation

```
g = trades.groupby('sym')
g['qty'].sum()
g[['px', 'qty']].mean()
g.agg(['min', 'max', 'mean'])
g['px'].ohlc()                                    # open, high, low, close
per group

# Named aggregation: flat, explicit output columns
summary = trades.assign(notional=trades['px'] *
trades['qty']).groupby('sym').agg(
    trades=('px', 'size'),
    volume=('qty', 'sum'),
    notional=('notional', 'sum'),
    high=('px', 'max'),
    low=('px', 'min'),
    last=('px', 'last'),
)
summary['vwap'] = summary['notional'] / summary['volume']

trades.groupby('sym', as_index=False)['qty'].sum()    # keys stay as columns
```

| Method | Counts or returns |
|---|---|
| `size()` | rows per group, including NaN |
| `count()` | non-null values per column |
| `nunique()` | distinct values |

| Method | Counts or returns |
|---|---|
| `first()` / `last()` | first / last NON-NULL value per column |
| `nth(0)` / `nth(-1)` | the actual first / last row |
| `head(n)` / `tail(n)` | first / last n rows, original index kept |
| `idxmax()` / `idxmin()` | index label of the extreme |
| `quantile(q)`, `median()`, `std()`, `var()`, `sem()`, `prod()` | statistics |

## 7.3 transform: results aligned to rows

```
g = df.groupby('date')['ret']
df['ret_demeaned'] = df['ret'] - g.transform('mean')           # cross-
sectional demeaning
df['z'] = (df['ret'] - g.transform('mean')) / g.transform('std')
df['weight'] = df['mcap'] / df.groupby('date')['mcap'].transform('sum')
df['n_names'] = df.groupby('date')['sym'].transform('size')
df['vol_share'] = df['qty'] / df.groupby(['date', 'sym'])
['qty'].transform('sum')
```

Pass built-in names as strings (`'mean'`, `'sum'`) instead of lambdas: they run in optimized Cython instead of calling Python once per group.

## 7.4 filter: keep or drop whole groups

```
liquid = df.groupby('sym').filter(lambda x: x['qty'].sum() > 1_000_000)   #
Python call per group

mask = df.groupby('sym')['qty'].transform('sum') > 1_000_000               #
vectorized equivalent
liquid = df[mask]

full_days = df[df.groupby('date')['sym'].transform('nunique') >= 40]      #
dates with enough names
```

## 7.5 apply: the general fallback

```
def top_n(d, n=3):
    return d.nlargest(n, 'qty')

df.groupby('sym', group_keys=False)[['ts', 'px', 'qty']].apply(top_n)
```

`apply` calls Python once per group and must infer how to combine the pieces, so it is slow with many groups. Select only the columns the function needs (pandas 2.2 deprecated passing the grouping columns into `apply`), and rewrite with `agg` / `transform` whenever possible.

## 7.6 Per-group cumulative and positional operations

```
g = df.sort_values(['sym', 'ts']).groupby('sym')
df['ret'] = g['px'].pct_change()              # never mixes two symbols
df['prev_px'] = g['px'].shift(1)
df['cum_qty'] = g['qty'].cumsum()
df['trade_no'] = g.cumcount()                 # 0, 1, 2, ... within each
group
df['grp_id'] = g.ngroup()                     # integer id per group
df['peak'] = g['px'].cummax()
df['first_px_of_day'] = df.groupby(['sym', 'date'])['px'].transform('first')
```

The single most common panel bug is `df['px'].pct_change()` on a long table sorted by symbol: the first return of every symbol is computed from the previous symbol's last price. Always group first.

## 7.7 Rolling and expanding windows within groups

```
r = df.groupby('sym')['px'].rolling(20, min_periods=20).mean()
df['ma20'] = r.reset_index(level=0, drop=True)                # drop the
group level to realign

df['ma20'] = df.groupby('sym')['px'].transform(lambda x:
x.rolling(20).mean())   # simpler, slower
df['exp_max'] = df.groupby('sym')['px'].transform('cummax')
df['ewm_vol'] = df.groupby('sym')['ret'].transform(lambda x:
x.ewm(span=20).std())
```

## 7.8 Performance tips

- Use built-in reductions ( `sum` , `mean` , `std` , `size` ) and string names in `agg` / `transform` .

- Convert string keys to `category` (with `observed=True`) or integer codes before grouping very large frames.
- Pass `sort=False` when the order of groups does not matter.
- `engine='numba'` is available for `agg`, `transform` and rolling `apply` with numeric user functions.
- For millions of tiny groups, sorting once and using NumPy `reduceat` or Polars can be an order of magnitude faster.

## 8. Combining data: merge, join, concat and as-of joins

Use `merge` for SQL-style key joins, `concat` for stacking, and `merge_asof` for "latest value at or before this time" joins, which is the backbone of trade-quote alignment and point-in-time research. Always validate join cardinality; silent row explosion is the classic failure.

### 8.1 merge

```
pd.merge(trades, ref, on='sym', how='left')                          # keep
every trade
pd.merge(a, b, left_on='symbol', right_on='sym', how='inner')
pd.merge(a, b, left_index=True, right_index=True, how='outer')
pd.merge(a, b, on=['date', 'sym'], suffixes=('_ours', '_vendor'))
pd.merge(a, b, on='sym', how='left', indicator=True)                 # _merge:
left_only / right_only / both
pd.merge(a, b, on='sym', how='left', validate='many_to_one')          # raises
if ref has duplicate keys
pd.merge(dates, syms, how='cross')                                    # every
date x every symbol
pd.merge(a, b, on='sym', how='left_anti')                             #
pandas 3.0: rows of a with no match in b
```

| how | Keeps |
|---|---|
| `inner` | keys present on both sides |
| `left` / `right` | all keys of one side; NaN where the other has no match |
| `outer` | union of keys |
| `cross` | Cartesian product, no keys |
| `left_anti` / `right_anti` | rows of one side without a match (pandas 3.0) |

## 8.2 Row explosion and how to catch it

If a key appears m times on the left and n times on the right, the result has m times n rows for that key. A duplicated row in a reference file can silently double your PnL.

```
n_before = len(trades)
out = trades.merge(ref, on='sym', how='left', validate='many_to_one')
assert len(out) == n_before
assert ref['sym'].is_unique
out['_merge'].value_counts()      # with indicator=True: how many trades
found no reference row
```

## 8.3 join and concat

```
prices.join(volumes, how='inner')                          # index-on-index
merge
prices.join(ref.set_index('sym'), on='sym')                # column of the
left, index of the right

pd.concat([df_day1, df_day2], ignore_index=True)        # stack rows
(DataFrame.append was removed in 2.0)
pd.concat([px, vol], axis=1)                               # side by side,
aligned on the index
pd.concat([px, vol], axis=1, join='inner')                 # only common index
labels
pd.concat({'NSE': nse, 'BSE': bse}, names=['exch'])     # adds an outer index
level
pd.concat(frames, axis=1, sort=False)                      # be explicit about
sort with DatetimeIndex (3.0)
```

Collect pieces in a list and call `concat` once; concatenating inside a loop is quadratic.

## 8.4 merge_asof deep dive

`merge_asof` matches each left row with the nearest right row by an ordered key (usually time), optionally within groups.

```
trades = trades.sort_values('ts')        # BOTH frames must be sorted by the
`on` key
quotes = quotes.sort_values('ts')

tq = pd.merge_asof(
    trades, quotes,
    on='ts',                              # ordered key (datetime or numeric)
    by='sym',                             # exact match on symbol first
    direction='backward',                 # last quote with ts <= trade ts
(default)
    tolerance=pd.Timedelta('500ms'),      # ignore quotes older than 500 ms -
> NaN
    allow_exact_matches=False,            # strictly earlier quotes only
)
tq['mid'] = (tq['bid'] + tq['ask']) / 2
tq['side'] = np.sign(tq['px'] - tq['mid'])               # Lee-Ready style
classification
tq['eff_spread_bps'] = 2 * (tq['px'] - tq['mid']).abs() / tq['mid'] * 1e4
```

| Parameter | Meaning | Trading use |
|---|---|---|
| `direction='backward'` | last right row at or before | what was known at the time: safe |
| `direction='forward'` | first right row at or after | markouts (price 1 s after a fill) |
| `direction='nearest'` | closest either side | data reconciliation only; leaks future in research |
| `tolerance` | maximum gap | reject stale quotes |
| `allow_exact_matches=False` | strict inequality | avoid same-timestamp leakage |
| `by` / `left_by` / `right_by` | exact-match groups | symbol, venue |

Gotchas: sorting must be global on `on`, not per group; the key dtypes should match, so convert both sides to one unit with dt.as_unit (`datetime64[us]` vs `datetime64[ns]`, or naive vs tz-aware, can raise); NaN in the key is not allowed; with `by`, missing groups simply produce NaN.

Markouts with a forward as-of join:

```
fills['ts_1s'] = fills['ts'] + pd.Timedelta('1s')
m = pd.merge_asof(fills.sort_values('ts_1s'), quotes[['ts', 'sym',
'mid']].rename(columns={'ts': 'ts_1s', 'mid': 'mid_1s'}),
                  on='ts_1s', by='sym', direction='backward')
m['markout_bps'] = m['side'] * (m['mid_1s'] - m['px']) / m['px'] * 1e4
```

## 8.5 merge_ordered

```
pd.merge_ordered(spot, futures, on='ts', how='outer', fill_method='ffill')
# union of timestamps, each side carried forward
pd.merge_ordered(a, b, on='date', left_by='sym', fill_method='ffill')
# per group
```

## 8.6 Patching and comparing

```
primary.combine_first(backup)        # fill gaps in primary from backup,
aligned on index and columns
primary.update(corrections)          # overwrite in place with non-NA values
from corrections
vendor_a.compare(vendor_b)           # side-by-side differences (identically
labelled frames)
pd.testing.assert_frame_equal(a, b, check_exact=False, rtol=1e-9)
```

## 8.7 Point-in-time joins for reference and fundamental data

Join fundamentals on the date they became public, not the period they describe; joining quarterly results on quarter-end leaks numbers that were announced weeks later.

```
fund = fund.sort_values('announce_ts')               # columns: sym,
announce_ts, eps, ...
panel = panel.sort_values('ts')
pit = pd.merge_asof(panel, fund, left_on='ts', right_on='announce_ts',
by='sym', direction='backward')

# Index membership with effective dates: was the stock in NIFTY 50 on that
day?
membership = membership.sort_values('effective_from')          # sym,
effective_from, in_index
panel = pd.merge_asof(panel, membership, left_on='ts',
right_on='effective_from', by='sym')
universe = panel[panel['in_index'] == True]                   # avoids
survivorship bias
```

# 9. Reshaping

Reshaping moves data between long (one row per observation) and wide (one column per entity) layouts. `pivot` / `unstack` go long to wide, `melt` / `stack` go wide to long, and `pivot_table` aggregates while it pivots.

## 9.1 pivot and pivot_table

```
long = pd.DataFrame({
    'date': pd.to_datetime(['2026-10-19'] * 2 + ['2026-10-20'] * 2),
    'sym': ['INFY', 'TCS'] * 2,
    'close': [1500.0, 3900.0, 1510.0, 3880.0],
})
wide = long.pivot(index='date', columns='sym', values='close')   # requires
unique (date, sym) pairs
rets = wide.pct_change()                                          # per-
symbol returns in one call

pt = trades.pivot_table(index='date', columns='sym', values='qty',
aggfunc='sum', fill_value=0)
pt = trades.pivot_table(index='sym', values=['qty', 'px'], aggfunc={'qty':
'sum', 'px': 'mean'})
pt = trades.pivot_table(index='hour', columns='side', values='qty',
aggfunc='sum', margins=True)  # totals row/col
```

`pivot` raises `ValueError: Index contains duplicate entries` when a (row, column) pair repeats; use `pivot_table` with an explicit `aggfunc`, or deduplicate first.

## 9.2 Option chain layout

```
chain = pd.DataFrame({
    'strike': [24000, 24000, 24100, 24100],
    'type':   ['CE', 'PE', 'CE', 'PE'],
    'ltp':    [180.0, 95.0, 120.0, 135.0],
    'oi':     [1.2e6, 1.5e6, 0.9e6, 1.1e6],
})
side_by_side = chain.pivot(index='strike', columns='type', values=['ltp',
'oi'])
side_by_side.columns = [f'{v}_{t}' for v, t in side_by_side.columns]   #
flatten MultiIndex columns
# -> ltp_CE, ltp_PE, oi_CE, oi_PE per strike
side_by_side['pcr_oi'] = side_by_side['oi_PE'] / side_by_side['oi_CE']
```

## 9.3 melt (wide to long)

```
long_again = wide.reset_index().melt(id_vars='date', var_name='sym',
value_name='close')
features.melt(id_vars=['date', 'sym'], value_vars=['mom', 'vol', 'size'],
              var_name='factor', value_name='exposure')
```

## 9.4 stack and unstack

```
panel = long.set_index(['date', 'sym'])['close']
wide = panel.unstack('sym')                         # inner level -> columns
wide = panel.unstack('sym', fill_value=0)           # fill holes created by the
pivot
back = wide.stack()                                 # columns -> inner index
level
wide.stack().reset_index(name='close')              # straight to a long
DataFrame

multi_col = df.set_index(['date', 'sym'])[['close', 'volume']].unstack('sym')
# columns: (field, sym)
multi_col['close']                                   # one field, all symbols
multi_col.swaplevel(axis=1).sort_index(axis=1)      # columns: (sym, field)
```

## 9.5 crosstab, explode, dummies, wide_to_long

```
# How often does today's signal match tomorrow's direction?
pd.crosstab(df['signal'], np.sign(df['fwd_ret']), normalize='index')

fills = pd.DataFrame({'order': [1, 2], 'fill_qty': [[10, 5], [20]]})
fills.explode('fill_qty')                           # one row per list element

pd.get_dummies(df['sector'], prefix='sec', dtype=float)   # one-hot for
regressions

w = pd.DataFrame({'sym': ['A', 'B'], 'px2025': [1, 2], 'px2026': [3, 4]})
pd.wide_to_long(w, stubnames='px', i='sym', j='year')    # px2025, px2026 ->
(sym, year) rows

df.T                                                # transpose: rows <->
columns (mixed dtypes -> object)
```

### 9.6 Which layout for which job

| Task | Layout |
|---|---|
| Store and append daily data | long |
| Join with reference data (sector, lot size) | long |
| Cross-sectional rank or z-score per date | wide (`rank(axis=1)`) or long with `groupby('date')` |
| Correlation matrix, PCA, portfolio returns | wide |
| Per-symbol time-series features | long with `groupby('sym')`, or wide column-wise |
| Plotting many series | wide |

## 10. Time series

Store timestamps in UTC, convert to exchange time (IST) for session logic, keep the index sorted, and be explicit about bar boundaries (`label`, `closed`, `offset`). Most time-series bugs are off-by-one-bar errors or bars aligned to midnight instead of the 09:15 open.

### 10.1 Timestamps, Timedeltas and parsing

```
pd.Timestamp('2026-10-19 09:15:00', tz='Asia/Kolkata')
pd.Timestamp.now('UTC')
pd.Timedelta('1min 30s'); pd.Timedelta(microseconds=250)
pd.to_datetime(df['ts'], format='%Y-%m-%d %H:%M:%S.%f')        # explicit
format: fast and unambiguous
pd.to_datetime(df['epoch_ns'], unit='ns', utc=True)             # integer
feeds
pd.to_datetime(df['date'], format='%d-%b-%Y')                   # 19-Oct-2026
(common in NSE bhavcopy files)
pd.to_datetime(df['ts'], errors='coerce')                       # bad strings
-> NaT
```

## 10.2 Time zones

```
idx = pd.DatetimeIndex(df['ts'])
idx = idx.tz_localize('UTC')                  # naive -> aware: declares what
the stamps already mean
idx_ist = idx.tz_convert('Asia/Kolkata')      # aware -> another zone: same
instant, new wall clock
idx_ist.tz_localize(None)                     # drop the zone (keeps IST
wall-clock values)
```

- `tz_localize` attaches a zone without changing the wall-clock time; `tz_convert` changes the wall-clock time without changing the instant.
- IST has no daylight saving, but global feeds (CME, LSE) do: use `ambiguous=` and `nonexistent=` when localizing their local times.
- pandas 3.0 returns standard-library `zoneinfo` objects instead of `pytz`.
- You cannot compare or join naive and aware timestamps; pick one convention per pipeline.

## 10.3 Ranges and frequency aliases

```
pd.date_range('2026-10-01', '2026-10-31', freq='B')                 #
weekdays
pd.date_range('2026-10-19 09:15', '2026-10-19 15:30', freq='1min')  # one
session of minutes
pd.date_range('2026-01-31', periods=12, freq='ME')                  # month
ends
pd.bdate_range('2026-10-01', '2026-10-31', freq='C', holidays=nse_holidays)
```

| Alias | Meaning | Alias | Meaning |
|---|---|---|---|
| `D` | calendar day | `ME` / `MS` | month end / start |
| `B` | business day | `BME` / `BMS` | business month end / start |
| `C` | custom business day | `QE` / `YE` | quarter end / year end |
| `W-FRI` | weekly, anchored on Friday | `h` | hour |
| `min` | minute | `s`, `ms`, `us`, `ns` | second and sub-second |

Old aliases `M`, `Q`, `Y`, `BM` were removed in pandas 3.0 (use `ME`, `QE`, `YE`, `BME`); uppercase `H`, `T`, `S`, `L`, `U`, `N` were deprecated in 2.2 in favour of `h`, `min`, `s`, `ms`, `us`, `ns`.

## 10.4 resample: bars from ticks

```
ticks = ticks.set_index('ts').sort_index()      # DatetimeIndex in IST
bars = ticks['px'].resample('1min').ohlc()
bars['volume'] = ticks['qty'].resample('1min').sum(min_count=1)
bars['vwap'] = (ticks['px'] * ticks['qty']).resample('1min').sum() /
bars['volume']
bars['n_trades'] = ticks['px'].resample('1min').count()

bars = ticks.resample('5min').agg(
    open=('px', 'first'), high=('px', 'max'), low=('px', 'min'), close=('px',
'last'),
    volume=('qty', 'sum'),
)
```

Bar conventions:

| Parameter | Default (minute bars) | Effect |
|---|---|---|
| `closed='left'` | bin is `[09:15, 09:16)` | a tick at exactly 09:16:00 goes to the next bar |
| `label='left'` | bar stamped 09:15 | stamp with `label='right'` to mark when the bar is complete |
| `origin='start_day'` | bins counted from midnight | fine for 1, 5, 15 minute bars |
| `offset='15min'` | shifts bin edges | hourly bars at 09:15, 10:15, ... |

```
hourly = ticks['px'].resample('1h', offset='15min').ohlc()          # 09:15-
10:15, 10:15-11:15, ...
# 75-minute bars anchored at 09:15 EVERY day (1440 is not a multiple of 75,
so offset alone drifts across days)
open_ = ticks.index.normalize() + pd.Timedelta('9h15min')
k = (ticks.index - open_) // pd.Timedelta('75min')
h75 = ticks.groupby(open_ + k * pd.Timedelta('75min'))['px'].ohlc()
```

Label choice matters for joins: a left-labelled 09:15 bar contains information up to 09:15:59.999. Joining it to a 09:15 signal timestamp leaks almost a minute of the future. Use `label='right'` (or shift by one bar) when bars feed into signals.

Empty bins appear as NaN rows (OHLC) or 0 (`sum`); forward-fill the close for empty bars and set their volume to 0 explicitly.

## 10.5 Upsampling and frequency conversion

```
daily.resample('1h').ffill()                    # upsample by repeating the
last value
sparse.asfreq('1min')                           # reindex to a fixed grid, no
aggregation
sparse.asfreq('1min', method='ffill')
quotes['mid'].resample('1s').last().ffill()   # 1-second snapshot grid from
irregular quotes
```

## 10.6 Shifting in rows vs in time

```
s.shift(1)                     # data moves down one ROW; index unchanged
s.shift(1, freq='1min')        # INDEX moves forward one minute; data
unchanged
s.shift(-1)                    # leads: tomorrow's value at today's row
(labels only, never features)
s.diff(); s.pct_change(5)      # per row

# Value from exactly 5 minutes earlier on an irregular index (as-of in time,
not rows)
lag = s.copy(); lag.index = lag.index + pd.Timedelta('5min')
prev_5m = pd.merge_asof(s.rename('now').to_frame(),
lag.rename('then').to_frame(),
                        left_index=True, right_index=True)
```

## 10.7 Sessions, calendars and expiries

```
session = ticks.between_time('09:15', '15:30')                # inclusive by
default
pre_open = ticks.between_time('09:00', '09:08')
first_15 = ticks.between_time('09:15', '09:29:59.999')
ticks[ticks.index.dayofweek < 5]                              # weekdays

day = ticks.index.date
open_px = ticks.groupby(day)['px'].transform('first')
ticks['ret_from_open'] = ticks['px'] / open_px - 1

from pandas.tseries.offsets import CustomBusinessDay, LastWeekOfMonth
nse_holidays = pd.to_datetime(['2026-10-02', '2026-11-09'])   # placeholder
list: load the official one
nse_bday = CustomBusinessDay(holidays=nse_holidays)
pd.date_range('2026-10-01', '2026-10-31', freq=nse_bday)
pd.Timestamp('2026-10-01') + 3 * nse_bday                      # three
trading days later
```

```
def monthly_expiry(year, month, weekday, holidays):
    """Last given weekday of the month, moved back to the previous trading
day if it is a holiday."""
    d = pd.Timestamp(year, month, 1) + LastWeekOfMonth(weekday=weekday)
    while d in set(holidays) or d.dayofweek >= 5:
        d -= pd.Timedelta(days=1)
    return d
```

Exchanges change expiry weekdays and holiday lists; keep both as versioned reference data rather than hard-coding them.

### 10.8 Periods

```
monthly = daily_ret.groupby(daily_ret.index.to_period('M')).apply(lambda r:
(1 + r).prod() - 1)
monthly = (1 + daily_ret).resample('ME').prod() - 1           # same, with
month-end timestamps
p = pd.Period('2026-10', freq='M'); p.start_time, p.end_time, p + 1
```

### 10.9 Time-weighted averages on irregular data

```
q = quotes.set_index('ts').sort_index()
dur = q.index.to_series().diff().shift(-1).dt.total_seconds()    # how long
each quote was live
tw_spread = (q['spread'] * dur).sum() / dur.sum()                # time-
weighted average spread
tw_by_min = (q['spread'] * dur).resample('1min').sum() /
dur.resample('1min').sum()
```

A simple mean of quotes over-weights bursts of rapid updates; a time-weighted mean reflects what a passive trader actually faced.

## 11. Window functions: rolling, expanding, ewm

Windows are trailing and include the current row by default, so a value at time t uses data up to and including t. Count-based windows ( `rolling(20)` ) suit regular bars; time-based windows ( `rolling('5min')` ) suit irregular ticks.

## 11.1 rolling

```
r = df['ret']
r.rolling(20).mean()                          # NaN until 20 observations
exist
r.rolling(20, min_periods=10).std()           # start emitting after 10
r.rolling(20).agg(['mean', 'std', 'min', 'max'])
r.rolling(20).quantile(0.05); r.rolling(20).median(); r.rolling(20).skew()
r.rolling(20).rank(pct=True)                  # percentile of today within its
window
r.rolling(20, closed='left').mean()           # window EXCLUDES the current
row: features for row t from t-20..t-1
r.rolling(20, center=True).mean()             # uses the future: smoothing for
plots only
r.rolling(20, step=5).mean()                  # evaluate every 5th row
r.rolling(10, win_type='gaussian').mean(std=3)   # weighted windows (needs
SciPy)
```

| Parameter | Default | Notes |
|---|---|---|
| `window` | required | int = number of rows; offset string = time span |
| `min_periods` | window size (int); 1 (time) | lower it to get early values |
| `center` | False | True leaks future data |
| `closed` | `'right'` | `'left'` excludes the current row; `'both'`, `'neither'` |

## 11.2 Rolling covariance, correlation and beta

```
beta = r_stock.rolling(60).cov(r_index) / r_index.rolling(60).var()
corr = r_stock.rolling(60).corr(r_index)
pair_corr = rets.rolling(60).corr()           # DataFrame: MultiIndex (date,
sym) x sym, every pair
pair_corr.loc['2026-10-19']                   # the 60-day correlation matrix
on that date
```

## 11.3 Time-based windows on ticks

```
t = ticks.set_index('ts').sort_index()        # index must be monotonic
t['vol_5m'] = t['qty'].rolling('5min').sum()                          #
trailing 5 minutes, variable count
t['vwap_5m'] = (t['px'] * t['qty']).rolling('5min').sum() / t['vol_5m']
t['n_trades_1s'] = t['px'].rolling('1s').count()                      # burst
detection
t['vol_prior_5m'] = t['qty'].rolling('5min', closed='left').sum()   #
excludes the current trade
```

## 11.4 expanding

```
r.expanding().mean()                          # all history up to t
r.expanding(min_periods=60).std()
px.expanding().max()                          # running peak (same as cummax)
z = (r - r.expanding(252).mean()) / r.expanding(252).std()    # walk-forward
normalization, no look-ahead
```

## 11.5 ewm: exponentially weighted windows

```
r.ewm(span=20, adjust=False).mean()          # classic recursive EMA: y_t = a
x_t + (1 - a) y_{t-1}
r.ewm(halflife=10).std()                      # EWMA volatility
r.ewm(alpha=0.06, adjust=False).var()        # RiskMetrics-style (lambda =
0.94)
r.ewm(span=20).corr(other)                    # EW correlation
ticks['px'].ewm(halflife='30s', times=ticks['ts']).mean()   # irregular
timestamps: decay by elapsed time
```

| Parameter | Relation to the smoothing factor alpha |
|---|---|
| `span` = s | alpha = 2 / (s + 1) |
| `com` = c | alpha = 1 / (1 + c) |
| `halflife` = h | alpha = 1 - exp(-ln 2 / h) |
| `alpha` | given directly, 0 < alpha <= 1 |

- `adjust=True` (default) divides by the sum of weights, which removes the start-up bias; `adjust=False` is the textbook recursion and matches most trading platforms. State which one you use.

- `ignore_na=False` (default) lets gaps decay the weights by elapsed rows; `True` ignores them.
- `min_periods` suppresses the noisy first values.

## 11.6 Custom and forward-looking windows

```
from pandas.api.indexers import FixedForwardWindowIndexer
fwd = FixedForwardWindowIndexer(window_size=5)
df['fwd_max_5'] = df['px'].rolling(fwd, min_periods=1).max()   # max of the
NEXT 5 rows incl. current: a label only

from pandas.api.indexers import VariableOffsetWindowIndexer
idx = VariableOffsetWindowIndexer(index=df.index, offset=pd.offsets.BDay(5))
df['sum_5bd'] = df['qty'].rolling(idx).sum()                    # five
business days back
```

Forward windows are legitimate for building targets (for example, the maximum adverse excursion over the next 5 bars) but must never feed features.

## 11.7 Performance and precision

- Built-in window methods (`mean`, `sum`, `std`, `min`, `max`, `count`) use O(n) online algorithms in Cython.
- `rolling(...).apply(func)` calls Python per window. Use `raw=True` (NumPy arrays instead of Series) and `engine='numba'` to compile it; `method='table'` applies one Numba function across several columns at once.
- Online rolling sums and variances accumulate floating-point error over very long series with large values; centre the data (for example, work in returns rather than price levels) when precision matters.

```
def range_frac(x):                    # position of the last value inside
the window range
    lo, hi = x.min(), x.max()
    return (x[-1] - lo) / (hi - lo) if hi > lo else 0.5

df['stoch'] = df['px'].rolling(14).apply(range_frac, raw=True,
engine='numba')
```

## 11.8 Alignment rule for signals

A rolling value stamped t includes the observation at t. If you trade at t using a bar that only completes at t, the feature is fine; if the signal must be known before t (for example, entry at the bar's open), use `shift(1)` or `closed='left'`.

# 12. Performance and memory

pandas is fast when work stays inside its vectorized C/Cython paths and slow whenever it calls Python per row or per group. Memory, not CPU, is usually the first limit: a common rule of thumb is to have several times the dataset's size in RAM because operations create intermediate copies.

## 12.1 The speed ladder

| Rank | Technique | Typical relative speed |
|---|---|---|
| 1 | Column arithmetic, built-in methods, NumPy ufuncs on `.to_numpy()` | fastest |
| 2 | `groupby` / `rolling` / `resample` with built-in aggregations | fast |
| 3 | `map` with a dict or Series, `merge`, `np.select` | fast |
| 4 | Numba (`engine='numba'` or `@njit` on arrays) | fast for custom logic |
| 5 | `apply` on columns, `groupby.apply`, `transform(lambda)` | slow |
| 6 | `apply(axis=1)`, `itertuples` | very slow |
| 7 | `iterrows`, appending rows in a loop | slowest |

```
# Slow: Python function per row
df['notional'] = df.apply(lambda r: r['px'] * r['qty'] if r['side'] == 'B'
else -r['px'] * r['qty'], axis=1)

# Fast: vectorized
sign = np.where(df['side'] == 'B', 1, -1)
df['notional'] = sign * df['px'] * df['qty']
```

On a million rows the vectorized version is typically a few hundred times faster.

## 12.2 Memory reduction checklist

1. Read only the columns you need (`usecols=` for CSV, `columns=` for Parquet).
2. Give dtypes at read time (`dtype={'qty': 'int32', 'sym': 'category'}`) so pandas never materializes wide defaults.
3. Convert repeated strings to `category`; let pandas 3.0 use Arrow-backed `str` for the rest.

4. Downcast floats and ints where precision and range allow.
5. Filter rows early (Parquet `filters=` push the filter into the reader).
6. Delete intermediates (`del tmp`) and avoid keeping several versions of a big frame alive.
7. Process in chunks or per partition (per day, per symbol) and combine small results.

## 12.3 Chunked processing

```
agg = []
for chunk in pd.read_csv('ticks_2026.csv', usecols=['ts', 'sym', 'px',
'qty'],
                         dtype={'sym': 'category', 'px': 'float64', 'qty':
'int64'},
                         chunksize=2_000_000):
    chunk['notional'] = chunk['px'] * chunk['qty']
    agg.append(chunk.groupby('sym', observed=True)[['qty',
'notional']].sum())
result = pd.concat(agg).groupby(level=0).sum()          # combine partial
sums
result['vwap'] = result['notional'] / result['qty']
```

Only decomposable aggregations (sum, count, min, max) combine this way. Means need sums and counts; medians and quantiles need a different approach (Polars, DuckDB, or approximate sketches).

## 12.4 eval and query

```
df.eval('notional = px * qty', inplace=True)                # numexpr:
multithreaded, fewer temporaries
df.eval('mid = (bid + ask) / 2')
df.query('px > 100 and qty >= 10 and sym in @watchlist')
```

`eval` / `query` help on large frames (hundreds of thousands of rows or more) with compound expressions; on small frames the parsing overhead makes them slower than plain indexing.

## 12.5 Structural habits that keep pandas fast

- Sort once (`sort_values(['sym', 'ts'])`) and set the index once; repeated sorts dominate many pipelines.
- Build columns in a dict and concatenate once instead of inserting hundreds of columns (fragmentation).
- Collect frames in a list and call `pd.concat` once; never grow a frame inside a loop.

- Parse datetimes with an explicit `format=` or read them from a binary format that already stores timestamps.
- Use integer or categorical keys for joins and groupbys on large data.
- Keep `numexpr` and `bottleneck` installed: pandas uses them automatically for some operations.

## 12.6 Numba inside pandas

```
df['x'].rolling(50).apply(my_func, raw=True, engine='numba')
df.groupby('sym')['ret'].transform(my_func, engine='numba')        #
func(values, index) signature
df.groupby('sym')['ret'].agg(my_func, engine='numba')

from numba import njit
@njit
def order_book_replay(side, px, qty): ...
result = order_book_replay(df['side'].to_numpy(), df['px'].to_numpy(),
df['qty'].to_numpy())
```

For path-dependent logic, extracting NumPy arrays and calling an `@njit` function is usually simpler and faster than any pandas construct.

## 12.7 Profiling

```
%timeit df.groupby('sym')['qty'].sum()
%prun -l 15 pipeline(df)                # function-level hot spots
%load_ext line_profiler
%lprun -f build_features build_features(df)
df.memory_usage(deep=True).sum() / 1e6   # MB
import tracemalloc                      # peak memory of a block of code
```

## 12.8 Usual suspects when pandas is slow

| Symptom | Likely cause | Fix |
|---|---|---|
| `read_csv` takes minutes | type inference, date parsing | `dtype=`, `usecols=`, `engine='pyarrow'`, or switch to Parquet |
| `groupby` is slow | `apply`/lambda, string keys | built-in aggs, `category` keys, `sort=False` |
| String filtering is slow | regex on object dtype | `regex=False`, Arrow-backed strings |

| Symptom | Likely cause | Fix |
|---|---|---|
| Memory spikes | many intermediate copies | chunking, `del`, fewer chained temporaries |
| `merge` is slow or huge | many-to-many keys | `validate=`, deduplicate, integer keys |
| `PerformanceWarning: highly fragmented` | column-by-column inserts | `pd.concat(axis=1)` once |

### 12.9 When to leave pandas

Move the heavy step to Polars, DuckDB or PyArrow when data no longer fits comfortably in memory, when you need multi-core execution for joins and groupbys, or when a lazy query plan (filter and project before reading) would avoid loading most of the data. Section 15 compares the options.

## 13. The pandas I/O API

The options below are the ones that change correctness or speed for market data: explicit dtypes and date formats, column and row pruning, compression, and partitioning. Section 14 covers which format to choose.

### 13.1 read_csv

```
bhav = pd.read_csv(
    'bhavcopy_20261019.csv.zip',          # compression inferred from the
extension (.gz, .zip, .zst, .bz2, .xz)
    usecols=['SYMBOL', 'SERIES', 'OPEN', 'HIGH', 'LOW', 'CLOSE', 'TOTTRDQTY',
'TIMESTAMP'],
    dtype={'SYMBOL': 'category', 'SERIES': 'category', 'TOTTRDQTY': 'int64'},
    parse_dates=['TIMESTAMP'], date_format='%d-%b-%Y',
    na_values=['-', 'NA'], keep_default_na=True,
    thousands=',',
)

ticks = pd.read_csv('ticks.csv', engine='pyarrow', dtype_backend='pyarrow')
# multithreaded parser
head = pd.read_csv('huge.csv', nrows=1000)
# inspect before a full load
for chunk in pd.read_csv('huge.csv', chunksize=1_000_000): ...
```

| Option | Why it matters |
|---|---|
| `usecols` | skip parsing columns you do not need |
| `dtype` | prevents inference (slow, can guess wrong, triggers `DtypeWarning` on mixed chunks) |
| `parse_dates` + `date_format` | fast, unambiguous timestamps |
| `index_col` | set the index while reading |
| `na_values`, `keep_default_na` | control which strings mean missing ("NA" can be a ticker!) |
| `engine='pyarrow'` | parallel parsing; supports fewer options |
| `chunksize` / `iterator=True` | bounded memory |
| `on_bad_lines='warn'` | survive corrupt rows instead of failing |
| `compression` | read compressed files directly |

## 13.2 to_csv

```
df.to_csv('out.csv.gz', index=False, float_format='%.4f', date_format='%Y-%m-%d %H:%M:%S.%f')
```

CSV loses dtypes, time zones, categoricals and exact float values unless you format carefully; use it for hand-off, not for storage you will read back repeatedly.

## 13.3 Parquet

```
df.to_parquet('trades_20261019.parquet', engine='pyarrow',
compression='zstd', index=False)

df.to_parquet('trades/', partition_cols=['date'], compression='zstd')    #
hive layout: trades/date=2026-10-19/xxx.parquet

sub = pd.read_parquet(
    'trades/',
    columns=['ts', 'sym', 'px', 'qty'],                                  #
column pruning
    filters=[('date', '>=', '2026-10-01'), ('sym', 'in', ['TCS', 'INFY'])],
# partition + row-group pruning
    dtype_backend='pyarrow',
)

import pyarrow.parquet as pq
pq.write_table(pa_table, 'f.parquet', compression='zstd',
row_group_size=1_000_000)   # control row groups
pq.ParquetFile('f.parquet').metadata                                     # row
groups, stats, schema
```

Notes: Parquet stores the pandas index in metadata unless `index=False`; timestamps keep their unit and time zone; categoricals round-trip as dictionary-encoded columns.

## 13.4 Feather (Arrow IPC)

```
df.reset_index().to_feather('cache.feather', compression='lz4')   # needs a
default index
pd.read_feather('cache.feather', columns=['ts', 'px'])
```

Feather is the fastest round trip for a pandas or Arrow table on local disk: little encoding work, and uncompressed files can be memory-mapped.

## 13.5 HDF5

```
df.to_hdf('store.h5', key='ticks', format='table', data_columns=['sym'],
complib='blosc', complevel=5)
with pd.HDFStore('store.h5') as st:
    st.append('ticks', new_day, data_columns=['sym'])
    sub = st.select('ticks', where='sym == "TCS" & index >= "2026-10-19
09:15"')
```

`format='fixed'` is faster but cannot be queried or appended; `format='table'` supports `where` and appends. HDF5 needs PyTables, is awkward with concurrent writers, and is mostly found in legacy research stacks.

## 13.6 Pickle

```
df.to_pickle('tmp.pkl'); pd.read_pickle('tmp.pkl')
```

Pickle preserves everything and is fast, but loading a pickle can execute arbitrary code and files may not load across pandas versions. Use it only for short-lived, trusted, local caches.

## 13.7 SQL databases

```
from sqlalchemy import create_engine
eng = create_engine('postgresql+psycopg://user:pw@host/db')
df = pd.read_sql('SELECT ts, sym, px FROM trades WHERE trade_date = %(d)s',
eng,
                 params={'d': '2026-10-19'}, parse_dates=['ts'])
for chunk in pd.read_sql('SELECT * FROM trades', eng, chunksize=500_000): ...
df.to_sql('daily_bars', eng, if_exists='append', index=False, method='multi',
chunksize=10_000)
```

Row-by-row inserts are slow; use `method='multi'`, the database's bulk loader (Postgres `COPY`), or an ADBC driver connection, which moves data in Arrow format. Always pass parameters with `params=` rather than formatting SQL strings.

## 13.8 JSON

```
events = pd.read_json('orders.ndjson', lines=True)                  # one
JSON object per line (logs, websockets)
flat = pd.json_normalize(api_payload, record_path='data', meta=['symbol',
'exchange'])
df.to_json('out.json', orient='records', date_format='iso')
```

### 13.9 Excel and others

```
pd.read_excel('positions.xlsx', sheet_name=None, engine='calamine')   # dict
of all sheets; calamine is fast
df.to_excel('report.xlsx', sheet_name='pnl', index=False)

pd.read_orc('f.orc'); pd.read_fwf('fixed_width.txt'); pd.read_html(url);
pd.read_clipboard()
pd.read_iceberg(...); df.to_iceberg(...)          # Apache Iceberg tables
(pandas 3.0)
pd.read_parquet('s3://bucket/trades/', storage_options={'anon': False})   #
cloud paths via fsspec
```

## 14. File formats for financial data

For quant research the default answer is: Parquet (zstd, partitioned by date, sorted by symbol and timestamp) for anything stored, Arrow IPC/Feather for fast local caches, CSV only for hand-off, and a purpose-built tick database (kdb+, ClickHouse, ArcticDB, or DuckDB over Parquet) when many people query the same history. The reasons are columnar layout, real types, compression, and the ability to read only the columns and rows you need.

### 14.1 Row-oriented vs column-oriented

- **Row-oriented** (CSV, JSON, Avro, database rows): each record is stored together. Good for appending events one at a time and reading whole records.
- **Column-oriented** (Parquet, ORC, Arrow, kdb+ splayed tables): each column is stored together. Good for analytics that touch a few columns of many rows ("average spread for TCS in October"), and similar values compress far better.

A query that needs 3 of 40 columns reads roughly 3/40 of a columnar file, but all of a CSV.

### 14.2 The comparison table

| Format | Layout | Typed schema | Compression | Read only some columns / rows | Append | Read speed | Ecosystem | Best use |
|---|---|---|---|---|---|---|---|---|
| CSV / TSV | row, text | no | external (gzip, zstd) | no / no | yes (append lines) | slow (parsing) | universal | hand-off, vendor files, small data |
| JSON / NDJSON | row, text | no | external | no / no | yes (NDJSON) | slow | universal | API payloads, logs |

| Format | Layout | Typed schema | Compression | Read only some columns / rows | Append | Read speed | Ecosystem | Best use |
|---|---|---|---|---|---|---|---|---|
| Excel | row, binary XML | weak | built in | no / no | no | very slow | business users | reports only |
| Parquet | columnar | yes (rich logical types) | snappy, zstd, gzip, lz4, brotli | yes / yes (row-group stats, partitions) | new files, not in place | fast | Spark, DuckDB, Polars, Arrow, cloud | research data lake, long-term storage |
| Arrow IPC / Feather v2 | columnar, in-memory layout | yes | lz4, zstd, or none | yes / no | streaming format only | very fast; memory-mappable | Arrow ecosystem | local caches, process hand-off |
| ORC | columnar | yes | zlib, snappy, zstd, lz4 | yes / yes | new files | fast | Hive, Spark | Hadoop-era warehouses |
| Avro | row, binary | yes (schema in header, evolution rules) | deflate, snappy, zstd | no / no | yes | moderate | Kafka, streaming | event streams, message schemas |
| HDF5 (PyTables) | chunked arrays / tables | yes | blosc, zlib | partly / yes with `table` format | yes | fast | scientific Python | legacy research stores, numeric arrays |
| Zarr | chunked N-D arrays | yes | blosc, zstd | yes (chunks) | yes (new chunks) | fast, parallel, cloud-friendly | xarray, Dask | large numeric cubes (vol surfaces over time) |
| Pickle | Python object stream | Python types | optional | no / no | no | fast | Python only | short-lived trusted caches |
| Raw binary / structs | fixed-width records | external spec | none | via offsets | yes | fastest to write | custom | capture logs, feed recordings |
| Delta Lake / Iceberg | table format over Parquet | yes + schema evolution | Parquet codecs | yes / yes | ACID transactions | fast | Spark, Trino, DuckDB, pandas 3.0 (Iceberg) | shared, versioned data lakes |
| kdb+ (q) | columnar, splayed and date-partitioned | yes | built in | yes / yes | yes (intraday in memory, end-of-day writes) | extremely fast | q language, commercial | bank and HFT tick databases |

| Format | Layout | Typed schema | Compression | Read only some columns / rows | Append | Read speed | Ecosystem | Best use |
|---|---|---|---|---|---|---|---|---|
| ArcticDB | versioned columnar store on S3/LMDB | yes | built in | yes / yes (date ranges) | yes | fast | pandas-native API | DataFrame time-series database for research teams |
| ClickHouse / QuestDB / TimescaleDB | database tables | yes | built in | yes (SQL) | yes | fast | SQL | shared query service over ticks and bars |
| DuckDB | embedded analytical database | yes | built in | yes (SQL over Parquet too) | yes | fast | Python, Arrow, Parquet | single-machine SQL on research data |

## 14.3 Why CSV is risky for market data

- **No types:** every read re-infers dtypes; a column that is all integers on Monday and has one blank on Tuesday changes to float.
- **Precision:** floats written with default formatting may not round-trip exactly; nanosecond timestamps become strings.
- **Missing-value ambiguity:** the strings "NA" or "NULL" may be real tickers or codes and get parsed as missing.
- **Time zones:** usually absent; you must know the convention out of band.
- **Speed:** parsing text is CPU-heavy; a year of ticks can take far longer to parse than to read as Parquet.
- **Excel round trips** reformat dates and strip leading zeros from codes.

CSV is still the right choice for receiving vendor and exchange files (bhavcopies, contract notes) and handing small results to other teams. Convert to Parquet at the boundary.

## 14.4 Parquet internals

```
file.parquet
├── Row group 1 (e.g. ~1M rows)
|   ├── Column chunk: ts      -> pages (encoded + compressed) + min/max
statistics
|   ├── Column chunk: sym     -> dictionary page + data pages
|   ├── Column chunk: px
|   └── Column chunk: qty
├── Row group 2 ...
└── Footer: schema, row-group offsets, per-column statistics, key-value
metadata (pandas index, dtypes)
```

- **Row groups** are the unit of parallelism and skipping. Readers check each row group's min/max statistics and skip those that cannot match a filter (predicate pushdown).
- **Column chunks** let a reader fetch only the requested columns (projection pushdown).
- **Encodings** shrink data before compression: dictionary encoding for repeated values (symbols, exchange codes), run-length encoding for long runs, delta encoding for sorted integers such as timestamps, and byte-stream-split for floats.
- **Logical types** include timestamps with a unit and a UTC flag, decimals, dates, strings, lists and structs. Legacy INT96 timestamps come from older Spark and Impala writers; convert them on read.
- **Sorting matters:** if a file is sorted by (symbol, timestamp), each row group covers a narrow symbol range, so a filter on one symbol skips most of the file. Random order defeats the statistics.

## 14.5 Compression codecs

| Codec | Ratio | Write speed | Read speed | Typical choice |
|---|---|---|---|---|
| None | 1x | fastest | fastest (memory-map) | hot local caches on fast SSD |
| LZ4 | low to medium | very fast | very fast | Feather caches, intraday scratch |
| Snappy | medium | very fast | very fast | Parquet default in pandas/pyarrow |
| ZSTD | high, tunable by level | fast (slower at high levels) | fast | research archives, cloud storage (less I/O cost) |

| Codec | Ratio | Write speed | Read speed | Typical choice |
|---|---|---|---|---|
| GZIP / zlib | high | slow | moderate | maximum compatibility |
| Brotli | highest | slow | moderate | write-once archives |
| Blosc (HDF5, Zarr) | medium to high on numeric arrays | very fast | very fast | numeric chunked arrays |

Compression ratios on tick data depend heavily on sorting and encoding: sorted timestamps delta-encode extremely well, repeated symbols dictionary-encode to almost nothing, and float prices compress less. Measure on your own data.

## 14.6 Designing a Parquet tick store

1. **Partition by date** (and optionally by segment or exchange): `ticks/segment=FO/date=2026-10-19/part-0.parquet`. Most research queries filter by date range first.
2. **Do not partition by symbol** when there are thousands of instruments; it creates millions of tiny files. If needed, use a hash bucket of the symbol (for example, 32 buckets).
3. **Sort each file by (symbol, timestamp)** so row-group statistics prune symbol queries.
4. **Size files and row groups sensibly:** files of roughly 100 MB to 1 GB and row groups of about a million rows or a few hundred MB avoid both the small-files problem and oversized reads.
5. **Use explicit types:** timestamps as int64 nanoseconds or `timestamp[ns, tz=UTC]`, prices as float64 or int64 ticks, quantities as int64, symbols as dictionary-encoded strings.
6. **Compress with zstd** for storage; keep raw files immutable and version derived datasets.
7. **Record schema and lineage** (source, feed version, corrections applied) in key-value metadata or a catalog.

```
import pyarrow as pa, pyarrow.parquet as pq, pyarrow.dataset as ds

day = day.sort_values(['sym', 'ts'], kind='stable')
table = pa.Table.from_pandas(day, preserve_index=False)
pq.write_to_dataset(table, root_path='ticks', partition_cols=['date'],
compression='zstd')

dataset = ds.dataset('ticks', format='parquet', partitioning='hive')
tbl = dataset.to_table(columns=['ts', 'sym', 'px', 'qty'],
                       filter=(ds.field('date') >= '2026-10-01') &
(ds.field('sym') == 'TCS'))
df = tbl.to_pandas()

import duckdb
duckdb.sql("""
    SELECT sym, date, sum(px * qty) / sum(qty) AS vwap
    FROM read_parquet('ticks/*/*.parquet', hive_partitioning = true)
    WHERE date BETWEEN '2026-10-01' AND '2026-10-19'
    GROUP BY ALL
""").df()
```

## 14.7 A layered storage architecture

| Layer | Contents | Format | Notes |
|---|---|---|---|
| Capture | raw exchange packets, feed recordings | PCAP or raw binary logs | append-only, never modified; replayable |
| Normalized ticks | decoded trades, quotes, order-book events | Parquet, date-partitioned, zstd | the research source of truth |
| Derived data | bars, features, corporate-action-adjusted prices, universes | Parquet or ArcticDB | versioned; rebuildable from the layer below |
| Working caches | the slice a notebook or backtest is using | Feather / Arrow IPC, memory-mapped | disposable |
| Query service | shared interactive access | kdb+, ClickHouse, ArcticDB, or DuckDB over Parquet | chosen by team size, budget and latency needs |

| Layer | Contents | Format | Notes |
|---|---|---|---|
| Live trading | in-process state | in-memory C++/Rust structures, shared memory | files are not on the hot path |

## 14.8 Which format for which question

| Situation | Choose | Why |
|---|---|---|
| Store years of ticks for research | Parquet + zstd, partitioned by date | small, typed, prunable, readable by every tool |
| Reload the same intermediate frame many times today | Feather (lz4 or uncompressed) | fastest round trip, memory-mappable |
| Exchange end-of-day files from the exchange website | CSV in, Parquet out | convert once at ingestion |
| Share data between Python and a C++ service | Arrow IPC, or flat binary structs with a documented layout | zero-copy, no parsing |
| Event stream between services | Avro or Protobuf over Kafka, or SBE for low latency | schema evolution and compact rows |
| Many analysts querying the same ticks | kdb+, ClickHouse, ArcticDB, DuckDB/Parquet | concurrency and SQL/q access |
| Gridded numeric data (vol surfaces x time) | Zarr or HDF5 | chunked N-D arrays |
| A quick personal cache of Python objects | Pickle | only if trusted and short-lived |

## 14.9 Benchmark it yourself

```
import time, os
n = 5_000_000
rng = np.random.default_rng(0)
df = pd.DataFrame({
    'ts': pd.date_range('2026-10-19 09:15', periods=n, freq='5ms', tz='UTC'),
    'sym': pd.Categorical(rng.choice(['TCS', 'INFY', 'HDFCBANK', 'RELIANCE'],
n)),
    'px': rng.normal(1500, 5, n).round(2),
```

```
    'qty': rng.integers(1, 500, n),
})
writers = {
    'csv.gz': lambda p: df.to_csv(p, index=False),
    'parquet.snappy': lambda p: df.to_parquet(p, compression='snappy'),
    'parquet.zstd': lambda p: df.to_parquet(p, compression='zstd'),
    'feather.lz4': lambda p: df.to_feather(p, compression='lz4'),
    'pkl': lambda p: df.to_pickle(p),
}
readers = {
    'csv.gz': pd.read_csv, 'parquet.snappy': pd.read_parquet, 'parquet.zstd':
pd.read_parquet,
    'feather.lz4': pd.read_feather, 'pkl': pd.read_pickle,
}
for ext, w in writers.items():
    path = f'bench.{ext}'
    t0 = time.perf_counter(); w(path); t1 = time.perf_counter(); readers[ext]
(path); t2 = time.perf_counter()
    print(f'{ext:15s} size {os.path.getsize(path) / 1e6:8.1f} MB  write {t1 -
t0:6.2f}s  read {t2 - t1:6.2f}s')
```

Run this once on your machine before the interview; quoting your own measured numbers is far more convincing than quoting someone else's.

### 14.10 Interview answer

> For research storage I use Parquet: columnar, typed, compressed and readable by pandas, Polars, DuckDB and Spark. I partition by date, sort by symbol and timestamp so row-group statistics prune symbol queries, use zstd, and keep timestamps as int64 nanoseconds in UTC. For fast local reloads I use Feather/Arrow IPC, which can be memory-mapped. CSV is only for ingesting vendor files and handing off results, because it loses types and is slow to parse. When many users query the same history, a tick database such as kdb+, ClickHouse or ArcticDB, or DuckDB over the Parquet lake, sits on top.

## 15. pandas vs Polars vs DuckDB vs PyArrow vs Dask

Use pandas for interactive, index-aware work on data that fits in memory; Polars or DuckDB for heavy joins, groupbys and scans on large data; PyArrow as the storage and interchange layer; and Dask or Spark only when one machine is genuinely not enough. Because they all speak Arrow, moving between them is cheap.

## 15.1 Comparison

| | pandas | Polars | DuckDB | PyArrow | Dask / Spark |
|---|---|---|---|---|---|
| Model | eager DataFrame with an Index | eager or lazy DataFrame, query optimizer | embedded SQL analytical database | columnar arrays, compute kernels, dataset scanning | distributed or out-of-core DataFrames |
| Parallelism | mostly single-threaded | multithreaded by default | multithreaded | some kernels multithreaded | many cores, many machines |
| Larger than RAM | no (chunk manually) | streaming engine for lazy queries | yes, spills to disk | dataset scans in batches | yes |
| Index / labels | yes: alignment, MultiIndex | no index; explicit joins | no index; SQL | no index | pandas-like (Dask) |
| Time-series helpers | rich ( `resample` , offsets, calendars, `rolling('5min')` ) | good ( `group_by_dynamic` , `rolling` , `join_asof` ) | good ( `ASOF JOIN` , `time_bucket` , window functions) | basic | limited |
| Ecosystem | everything (statsmodels, scikit-learn, plotting) | growing fast | SQL tools, any Arrow source | storage layer for all of the others | big-data stacks |
| Weak spots | memory, speed on big data, single core | fewer niche features, different API | not a DataFrame API (but has a Python relational API) | low-level | overhead, complexity |

## 15.2 Moving data between them

```
import polars as pl, duckdb, pyarrow as pa

pl_df = pl.from_pandas(pdf)                 # via Arrow; zero-copy for many
dtypes
pdf2 = pl_df.to_pandas()

tbl = pa.Table.from_pandas(pdf, preserve_index=False)
pdf3 = tbl.to_pandas()
pdf4 = pd.DataFrame.from_arrow(tbl)          # pandas 3.0: any object
exposing the Arrow PyCapsule interface

res = duckdb.sql('SELECT sym, avg(px) AS avg_px FROM pdf GROUP BY sym').df()
# query a pandas frame by name
res = duckdb.sql('SELECT * FROM pl_df WHERE qty > 100').pl()
# results as Polars
```

## 15.3 The same as-of join three ways

```
# pandas
pd.merge_asof(trades.sort_values('ts'), quotes.sort_values('ts'), on='ts',
by='sym', direction='backward')

# Polars
trades_pl.sort('ts').join_asof(quotes_pl.sort('ts'), on='ts', by='sym',
strategy='backward')

# DuckDB
duckdb.sql('''
    SELECT t.*, q.bid, q.ask
    FROM trades t ASOF JOIN quotes q
      ON t.sym = q.sym AND t.ts >= q.ts
''').df()
```

## 15.4 The same 1-minute bars three ways

```
# pandas
ticks.set_index('ts').groupby('sym')['px'].resample('1min').ohlc()

# Polars
(ticks_pl.sort('sym', 'ts')
   .group_by_dynamic('ts', every='1m', group_by='sym')
   .agg(pl.col('px').first().alias('open'), pl.col('px').max().alias('high'),
        pl.col('px').min().alias('low'), pl.col('px').last().alias('close'),
        pl.col('qty').sum().alias('volume')))

# DuckDB
duckdb.sql('''
    SELECT sym, time_bucket(INTERVAL '1 minute', ts) AS bar,
           arg_min(px, ts) AS open, max(px) AS high, min(px) AS low,
arg_max(px, ts) AS close,
           sum(qty) AS volume
    FROM ticks GROUP BY ALL ORDER BY sym, bar
''').df()
```

## 15.5 Choosing in a quant workflow

| Task | Good default |
|---|---|
| Explore one day of ticks in a notebook | pandas |
| Build bars for a year of ticks across 2,000 instruments | Polars lazy scan or DuckDB over Parquet, then pandas for the result |
| Point-in-time joins on large panels | Polars `join_asof` or DuckDB `ASOF JOIN` |
| Factor research on a daily (date x symbol) panel | pandas (wide frames, `rank`, `corr`) |
| Statistical models | pandas + statsmodels / scikit-learn |
| Feed a C++ or GPU process | Arrow IPC, or NumPy arrays from any of them |
| Hundreds of GB, many machines | Spark or Dask, ideally reading the same Parquet lake |

GPU options (cuDF's pandas accelerator mode, Polars' GPU engine) can speed up large groupbys and joins when NVIDIA GPUs are available, with the same APIs.

### 15.6 Interview answer

> pandas is my default for interactive, label-aware work and anything downstream libraries expect. When the data is many gigabytes or the step is a big join or groupby, I push it to Polars' lazy engine or DuckDB over Parquet, which are multithreaded and can stream or spill, and bring the reduced result back to pandas. Arrow is the common memory format, so the hand-offs are cheap.

## 16. Quant workflows in pandas

These end-to-end recipes combine the earlier sections into the tasks a quant dev or data analyst actually does: clean ticks, build bars, adjust for corporate actions, compute factors, backtest, and process option chains.

### 16.1 Cleaning a day of ticks

```
def clean_ticks(raw: pd.DataFrame) -> pd.DataFrame:
    df = (raw
          .astype({'sym': 'category', 'px': 'float64', 'qty': 'int64'})
          .assign(ts=lambda d: pd.to_datetime(d['ts_ns'], unit='ns',
utc=True).dt.tz_convert('Asia/Kolkata'))
          .sort_values(['sym', 'ts', 'seq'], kind='stable')
          .drop_duplicates(subset=['sym', 'seq'])                       #
feed retransmissions
          .query('px > 0 and qty > 0'))
    df = df.set_index('ts')
    df = df.between_time('09:15', '15:30')                              #
continuous session only

    # Bad prints: far from a trailing rolling median of the same symbol
(live-safe, no centering)
    med = df.groupby('sym', observed=True)['px'].transform(lambda x:
x.rolling(51, min_periods=10).median())
    dev = (df['px'] / med - 1).abs()
    return df[(dev < 0.02) | med.isna()]
```

## 16.2 Bars and daily statistics per symbol

```
def make_bars(ticks, freq='1min'):
    g = ticks.groupby('sym', observed=True)
    bars = g['px'].resample(freq).ohlc()
    bars['volume'] = g['qty'].resample(freq).sum()
    bars['vwap'] = (ticks['px'] * ticks['qty']).groupby(ticks['sym'],
observed=True).resample(freq).sum() / bars['volume']
    bars['close'] = bars.groupby(level='sym')['close'].ffill()              #
empty bars: carry the close
    for c in ['open', 'high', 'low']:
        bars[c] = bars[c].fillna(bars['close'])
    return bars

daily = ticks.groupby(['sym', ticks.index.date], observed=True).agg(
    open=('px', 'first'), high=('px', 'max'), low=('px', 'min'), close=('px',
'last'),
    volume=('qty', 'sum'), trades=('px', 'size'))
```

## 16.3 Returns panel, forward returns and a universe mask

```
px = daily_long.pivot(index='date', columns='sym',
values='adj_close').sort_index()   # wide
ret = px.pct_change(fill_method=None)                    # no silent forward
fill through gaps
logret = np.log(px).diff()
fwd_1d = ret.shift(-1)                                   # label: next day's
return
fwd_5d = px.pct_change(5, fill_method=None).shift(-5)    # label: next 5
days

adv = (px * vol).rolling(20, min_periods=15).mean()      # average daily
traded value
universe = (adv > 5e7) & px.notna()                      # liquid names on
each date
ret_u = ret.where(universe.shift(1))                     # membership
decided with yesterday's data
```

## 16.4 Corporate-action adjustment

Splits and bonuses create artificial price gaps. Multiply all prices before the ex-date by the action's price factor (0.5 for a 2-for-1 split or 1:1 bonus) and divide volumes by it.

```
# actions: columns sym, ex_date, factor
f = (actions.pivot_table(index='ex_date', columns='sym', values='factor',
aggfunc='prod')
            .reindex(index=px.index, columns=px.columns).fillna(1.0))
cum = f[::-1].cumprod()[::-1].shift(-1).fillna(1.0)       # product of
factors with ex_date AFTER t
adj_px = px * cum
adj_vol = vol / cum

# Dividends for total returns: add the dividend on the ex-date
tot_ret = (px + divs.reindex_like(px).fillna(0)) / px.shift(1) - 1
```

Always adjust using information known at the time when building signals in a backtest (for example, recompute adjustments as of each date), or at least recognise that back-adjusted price levels embed future actions; returns are unaffected.

## 16.5 A cross-sectional factor study

```
mom = px.shift(21) / px.shift(252) - 1                          # 12-1 month
momentum
mom = mom.where(universe)
z = mom.sub(mom.mean(axis=1), axis=0).div(mom.std(axis=1), axis=0)    #
cross-sectional z-score
z = z.clip(-3, 3)

fwd = px.pct_change(21, fill_method=None).shift(-21)           # next-month
return (label)
ic = z.rank(axis=1).corrwith(fwd.rank(axis=1), axis=1)         # Spearman IC
per date
ic_monthly = ic.groupby(ic.index.to_period('M')).last()        # one non-
overlapping value per month
print(ic_monthly.mean(), ic_monthly.mean() / ic_monthly.std() * np.sqrt(12))
# mean IC and IC IR

q = z.rank(axis=1, pct=True)
quintile = np.ceil(q * 5)
q_ret = {k: fwd.where(quintile == k).mean(axis=1) for k in range(1, 6)}
q_ret = pd.DataFrame(q_ret)                                    # mean forward
return by quintile per date
long_short = q_ret[5] - q_ret[1]
```

Overlapping 21-day forward returns are autocorrelated; evaluate on non-overlapping monthly samples or adjust standard errors (Newey-West).

## 16.6 Rolling beta and residual returns

```
mkt = ret['NIFTY50']
stocks = ret.drop(columns='NIFTY50')
beta = stocks.rolling(120, min_periods=60).cov(mkt).div(mkt.rolling(120,
min_periods=60).var(), axis=0)
resid = stocks - beta.shift(1).mul(mkt, axis=0)              # beta estimated
with data up to yesterday
```

## 16.7 Vectorized portfolio backtest with monthly rebalancing

```
raw_w = z.div(z.abs().sum(axis=1), axis=0)                    # dollar-
neutral-ish, gross exposure 1
rebal_days = raw_w.groupby(raw_w.index.to_period('M')).tail(1).index   # last
trading day of each month
w = raw_w.loc[rebal_days].reindex(raw_w.index).ffill()         # hold weights
between rebalances

held = w.shift(1)                                              # weights set
at close t earn t -> t+1
gross = (held * ret).sum(axis=1, min_count=1)
turnover = w.diff().abs().sum(axis=1)
net = gross - turnover * 10 / 1e4                               # 10 bps per
unit of turnover

equity = (1 + net.fillna(0)).cumprod()
dd = equity / equity.cummax() - 1
stats = pd.Series({
    'ann_ret': equity.iloc[-1] ** (252 / len(net)) - 1,
    'ann_vol': net.std() * np.sqrt(252),
    'sharpe': net.mean() / net.std() * np.sqrt(252),
    'max_dd': dd.min(),
    'avg_turnover': turnover.mean(),
})
monthly = (1 + net).resample('ME').prod() - 1
```

### 16.8 Option chain processing

```
# chain: one row per (expiry, strike, type) with ltp, oi, iv; spot per
snapshot
chain['dist'] = (chain['strike'] - chain['spot']).abs()
atm = chain.loc[chain.groupby(['expiry', 'type'])['dist'].idxmin()]
straddle = atm.pivot(index='expiry', columns='type',
values='ltp').sum(axis=1)   # ATM straddle by expiry
atm_iv = atm.groupby('expiry')['iv'].mean()
# term structure

days = (pd.to_datetime(atm_iv.index) - pd.Timestamp('2026-10-19')).days
expected_move = straddle / chain['spot'].iloc[0]
# rough implied move to expiry

otm = chain[((chain['type'] == 'CE') & (chain['strike'] > chain['spot'])) |
            ((chain['type'] == 'PE') & (chain['strike'] < chain['spot']))]
smile = otm.pivot_table(index='strike', columns='expiry', values='iv')
# strike x expiry surface
pcr = chain.pivot_table(index='expiry', columns='type', values='oi',
aggfunc='sum').eval('PE / CE')
```

### 16.9 Position and PnL reconciliation

```
fills['signed_qty'] = np.where(fills['side'] == 'B', 1, -1) * fills['qty']
ours = fills.groupby('sym', observed=True)['signed_qty'].sum().rename('ours')
broker = broker_pos.set_index('sym')['net_qty'].rename('broker')
recon = pd.concat([ours, broker], axis=1).fillna(0)
breaks = recon[recon['ours'] != recon['broker']]

fills['cash'] = -fills['signed_qty'] * fills['px'] - fills['fees']
realized_and_open = fills.groupby('sym', observed=True).agg(cash=('cash',
'sum'), pos=('signed_qty', 'sum'))
realized_and_open['mtm'] = realized_and_open['cash'] +
realized_and_open['pos'] * last_px
```

## 17. pandas 2.x and 3.x changes worth knowing

pandas 3.0.0 was released on 21 January 2026, and the current patch release is 3.0.6 (17 September 2026). The changes most likely to come up in an interview, or to break existing research code, are the default `str` dtype, Copy-on-Write, and microsecond datetime resolution.

## 17.1 pandas 2.x highlights (2023 to 2025)

| Release | Change |
|---|---|
| 2.0 (Apr 2023) | Arrow-backed dtypes (`dtype_backend='pyarrow'`); non-nanosecond datetime resolutions; `Index` can hold any NumPy numeric dtype (no more `Int64Index` / `Float64Index`); `DataFrame.append` removed; Copy-on-Write available as an option |
| 2.1 (Aug 2023) | `applymap` renamed to `DataFrame.map`; `fillna(method=...)` deprecated in favour of `ffill` / `bfill`; new `stack` implementation |
| 2.2 (Jan 2024) | `Series.case_when`; frequency aliases `M`, `Q`, `Y` deprecated for `ME`, `QE`, `YE`; `H`, `T`, `S`, `L`, `U`, `N` deprecated for `h`, `min`, `s`, `ms`, `us`, `ns`; grouping columns in `groupby.apply` deprecated |
| 2.3 (Jun 2025) | bridge release: run your code on 2.3 without warnings before moving to 3.0 |

## 17.2 pandas 3.0 changes

| Area | What changed | What to do |
|---|---|---|
| Strings | text columns are inferred as the dedicated `str` dtype (PyArrow-backed when installed); it holds only strings; missing value is NaN | stop testing `dtype == object`; use `is_string_dtype` |
| Copy-on-Write | the default and only mode; every derived object behaves as a copy; chained assignment never works; `SettingWithCopyWarning` removed | write with one `loc` call on the target frame; drop defensive `.copy()` calls |
| Datetime resolution | parsing strings infers microseconds (nanoseconds only if the string has them); integer input keeps its unit | use `.dt.as_unit('ns')` before `astype('int64')` |
| Time zones | standard-library `zoneinfo` replaces `pytz` by default; `pytz` no longer required | adjust code that relies on `pytz` objects |

| Area | What changed | What to do |
|---|---|---|
| Frequency aliases | `M`, `Q`, `Y`, `BM`, … removed | `ME`, `QE`, `YE`, `BME` |
| `pd.col()` | expression syntax for `assign`, `loc`, getitem/setitem | `df.assign(n=pd.col('px') * pd.col('qty'))` |
| Arrow interchange | Arrow PyCapsule import/export; `DataFrame.from_arrow` | zero-copy hand-off to Polars, DuckDB |
| Joins | `how='left_anti'` / `'right_anti'` | anti joins without the `indicator` trick |
| I/O | `read_iceberg` / `to_iceberg` | Iceberg data lakes |
| GroupBy and windows | `skipna` on groupby reductions; `Rolling.first/last/nunique` | |
| `pd.offsets.Day` | now a calendar day (keeps time-of-day across DST) | matters for global markets, not IST |
| Nullable floats | NaN is treated as NA by default | |
| `Series[int]` | integer keys are always labels, never positions | use `.iloc` for positions |
| `inplace=True` methods | now return the object instead of `None` | prefer non-inplace style anyway |
| `copy=` keyword | deprecated on many methods (has no effect under CoW) | remove it |
| Requirements | Python 3.11+, NumPy 1.26+ | |

## 17.3 Migration checklist for a research codebase

1. Upgrade to pandas 2.3, run the full test suite with warnings as errors, and fix every `FutureWarning`.
2. Search for `dtype == object` and `select_dtypes('object')` used to find text columns.
3. Search for `astype('int64')` / `.view('int64')` on datetimes; add `.dt.as_unit('ns')`.
4. Search for chained assignment patterns (`df[...][...] =`) and `inplace=True` chains.
5. Replace old frequency strings (`'M'`, `'H'`, `'T'`) and `fillna(method=...)`.
6. Check timestamp units in Parquet and merges: mixing `us` and `ns` columns can produce surprises in joins.

Sources: pandas 3.0.0 release notes, pandas 3.0 announcement

# 18. Interview questions with model answers

Practise each answer aloud in 30 to 60 seconds, then add an example from your own work: tick data you have handled, a bug you found, or a pipeline you sped up.

## 18.1 Core concepts

1. **Series vs DataFrame vs Index?** A Series is one typed array with an Index; a DataFrame is a set of Series sharing a row Index, with column labels that are also an Index; an Index is an immutable, hashable set of labels used for lookup and alignment.
2. **What is index alignment?** Binary operations match rows (and columns) by label before computing; labels present on only one side give NaN. It prevents silent misalignment but produces unexpected NaN when indexes differ slightly.
3. **`loc` vs `iloc` vs `at`?** `loc` is label-based and its slices include the end; `iloc` is position-based and excludes the end; `at` / `iat` are fast single-value accessors.
4. **What is Copy-on-Write?** In pandas 3.0 every derived object behaves as a copy, with the real copy deferred until someone writes. Chained assignment like `df[mask]['x'] = 1` never changes `df`; write `df.loc[mask, 'x'] = 1`.
5. **Why is `iterrows` slow and what do you use instead?** It builds a Series per row and loses dtypes. Use vectorized column operations; for stateful loops, extract NumPy arrays and use Numba.
6. **`map` vs `apply` vs `transform` vs `agg`?** `map` is elementwise (or a dict/Series lookup); `apply` passes columns, rows or groups to a function and returns anything; `transform` must return the same shape as its input; `agg` reduces.
7. **How does a DataFrame store data?** Same-dtype NumPy columns are grouped in 2-D blocks; extension types (strings, categoricals, nullable, Arrow) are stored per column. Column operations are fast, row access is slow, and inserting many columns fragments the frame.
8. **NaN vs None vs `pd.NA` vs NaT?** NaN is the float missing value; None is converted to NaN or NA; `pd.NA` is the missing value of nullable and Arrow dtypes with three-valued logic; NaT is the missing datetime.
9. **Why did my integer column become float?** A NumPy integer column cannot hold NaN, so any missing value upcasts it to float64. Use `Int64` or an Arrow integer dtype.
10. **When do you use `category`?** For low-cardinality repeated labels (symbols, exchanges, sides): less memory, faster groupby and sorts. Watch out for unseen values, mismatched categories when concatenating, and use `observed=True`.

11. **What changed in pandas 3.0?** Default `str` dtype, Copy-on-Write only, microsecond default datetime resolution, `zoneinfo` time zones, removal of deprecated APIs and aliases, `pd.col` expressions, anti joins.

## 18.2 GroupBy, joins and reshaping

12. **`agg` vs `transform` vs `filter` vs `apply` in groupby?** `agg` gives one row per group; `transform` returns values aligned to the original rows (demeaning, group shares); `filter` keeps or drops whole groups; `apply` is a slow general fallback.
13. **`size` vs `count`; `first` vs `nth(0)`?** `size` counts rows including NaN; `count` counts non-null values per column. `first` returns the first non-null value per column; `nth(0)` returns the actual first row.
14. **How do you compute returns in a long (date, symbol) table?** Sort by symbol and time, then `df.groupby('sym')['px'].pct_change()`. Without the groupby, each symbol's first return is computed from the previous symbol's last price.
15. **How do you detect a bad merge?** Use `validate='many_to_one'` (or the right cardinality), compare row counts before and after, and use `indicator=True` to count unmatched rows.
16. **What does `merge_asof` need?** Both frames sorted by the `on` key, matching key dtypes, optional `by` for exact-match groups, a `direction`, and a `tolerance` to reject stale matches. Backward direction reflects what was known at the time.
17. **`join` vs `merge` vs `concat`?** `merge` joins on columns or indexes SQL-style; `join` is index-based merge shorthand; `concat` stacks objects along an axis, aligning the other axis.
18. **`pivot` vs `pivot_table`?** `pivot` only reshapes and fails on duplicate (index, column) pairs; `pivot_table` aggregates duplicates with `aggfunc` and can add margins.
19. **When do you use long vs wide data?** Long for storage, joins and per-symbol groupbys; wide (date x symbol) for cross-sectional math, correlations and portfolio returns.

## 18.3 Time series

20. **`resample` vs `groupby(pd.Grouper)` vs `rolling`?** `resample` buckets time into fixed bins and handles empty bins; `Grouper` does the same inside a multi-key groupby; `rolling` computes a trailing window for every row.
21. **What do `label` and `closed` do in `resample`, and why do they matter?** They decide which edge a bin includes and which edge stamps it. A left-labelled minute bar contains data up to the end of the minute, so joining it to a signal at its start time leaks the future.

22. **`shift(1)` vs `shift(1, freq='1min')`?** The first moves data down a row; the second moves the timestamps forward and leaves the data in place.
23. **`tz_localize` vs `tz_convert`?** `tz_localize` declares the zone of naive timestamps without changing the clock time; `tz_convert` changes the clock time of aware timestamps to another zone.
24. **`rolling(5)` vs `rolling('5min')`?** The first is the last 5 rows; the second is all rows in the trailing 5 minutes, a variable count, which suits irregular ticks.
25. **Explain `ewm` parameters.** `span`, `com`, `halflife` and `alpha` are four ways to set the decay; `adjust=True` normalizes weights to remove start-up bias, `adjust=False` is the recursive EMA; `times=` handles irregular timestamps.
26. **How do you build hourly bars that start at 09:15?** `resample('1h', offset='15min')`, or compute bar starts explicitly from minutes since the open for bar lengths that do not divide a day.
27. **How do you handle exchange holidays?** A `CustomBusinessDay` with an official holiday list, kept as versioned reference data, used for date ranges, offsets and expiry calculation.

## 18.4 Data quality and bias

28. **Give pandas operations that cause look-ahead bias.** `bfill`, `interpolate`, `rolling(center=True)`, `shift(-1)` used as a feature, full-sample normalization or winsorization, left-labelled bars joined at their start, joins on period-end instead of announcement dates.
29. **How do you avoid survivorship bias?** Build the universe as of each date from point-in-time membership and listing data (`merge_asof` on effective dates), including delisted names.
30. **What is the problem with forward-filling prices?** Stale prices create zero returns, understating volatility and correlation; fills across symbols or sessions leak data. Fill within symbol and session, with a limit, and track staleness.
31. **How do you handle duplicate or out-of-order ticks?** Sort stably by exchange timestamp and sequence number, drop exact duplicates by sequence, flag gaps in sequence numbers, and keep receive time separately from exchange time.
32. **How do you adjust for splits and bonuses?** Multiply prices before the ex-date by the cumulative product of later price factors and divide volumes by it; use total returns with dividends for performance.

## 18.5 Performance

33. **How do you make slow pandas code faster?** Profile, then vectorize (remove `apply`/loops), use built-in groupby and window methods, fix dtypes (categories, explicit date formats), avoid repeated sorts and concats, and move stateful logic to Numba.
34. **How do you reduce memory?** Read only needed columns, set dtypes at read time, categoricals for repeated strings, downcasting where safe, filtering early, chunking, and dropping intermediates.
35. **How would you process a 50 GB CSV?** Convert it once to partitioned Parquet with explicit types (chunked pandas, Polars `scan_csv`, or DuckDB), then query only the needed columns and partitions with Polars or DuckDB.
36. **When would you use Polars or DuckDB?** For data larger than comfortable memory, big joins and groupbys, multi-core speed, or lazy queries that filter before reading; then return the reduced result to pandas.
37. **When do `eval` and `query` help?** On large frames with compound arithmetic or filters, where numexpr avoids temporaries and uses several threads; on small frames they are slower.

## 18.6 File formats and storage

38. **CSV vs Parquet for market data?** CSV is untyped text: slow to parse, ambiguous missing values and precision loss. Parquet is columnar, typed, compressed and supports column and row-group pruning, so reads are smaller and faster.
39. **How does Parquet make queries fast?** Data is split into row groups and column chunks with min/max statistics; readers fetch only requested columns and skip row groups whose statistics cannot match the filter. Sorting the data makes the statistics selective.
40. **Snappy vs zstd vs lz4?** Snappy and lz4 are very fast with moderate compression (good for hot data and caches); zstd gives much better ratios at good speed (good for archives and cloud storage); gzip and brotli compress more but slowly.
41. **How would you partition tick data?** By date (and segment), sorted by symbol and timestamp within files, files of hundreds of MB, row groups around a million rows; not by symbol, which creates too many small files.
42. **Feather vs Parquet?** Feather (Arrow IPC) is the in-memory Arrow layout on disk: fastest to read and write and memory-mappable, best for local caches. Parquet is more compact, more widely supported, and has statistics for pruning: best for storage.
43. **Why not pickle?** Loading can execute arbitrary code, files can break across pandas versions, and other languages cannot read them.

44. **Where does HDF5 fit?** Fast chunked numeric storage with queryable tables, common in older research stacks, but weak for concurrent writers and cloud storage; Parquet or Zarr are usually better today.
45. **When would a team use kdb+, ArcticDB or ClickHouse?** When many users need fast, concurrent queries over years of ticks with intraday appends. kdb+ is the industry standard for tick data (commercial, q language); ArcticDB offers a pandas-native versioned store on object storage; ClickHouse offers SQL at scale.

# 19. Coding problems with solutions

Solve each one before reading the answer. Assume `trades` has columns `ts` (tz-aware IST), `date`, `sym`, `px`, `qty`, `side`, `seq`, and that `quotes` has `ts`, `sym`, `bid`, `ask`, `bid_qty`, `ask_qty`. Say the edge cases aloud: empty groups, NaN, ties, unsorted timestamps, and anything that could leak the future.

## 19.1 Warm-up

**1. Top 3 symbols by traded value each day.**

```
val = trades.assign(v=trades['px'] * trades['qty']).groupby(['date', 'sym'],
observed=True)['v'].sum()
top3 =
val.sort_values(ascending=False).groupby(level='date').head(3).sort_index()
```

**2. First and last trade time and price per symbol per day.**

```
summary = (trades.sort_values('ts', kind='stable')
                 .groupby(['date', 'sym'], observed=True)
                 .agg(first_ts=('ts', 'min'), first_px=('px', 'first'),
                      last_ts=('ts', 'max'), last_px=('px', 'last')))
```

**3. Monthly returns table (years as rows, months as columns).**

```
m = (1 + daily_ret).resample('ME').prod() - 1
table = pd.DataFrame({'year': m.index.year, 'month': m.index.month, 'ret':
m.to_numpy()}) \
          .pivot(index='year', columns='month', values='ret')
table['YTD'] = (1 + table.fillna(0)).prod(axis=1) - 1
```

**4. 20-day annualized volatility per symbol on a long table.**

```
d = daily.sort_values(['sym', 'date'])
d['ret'] = d.groupby('sym', observed=True)
['close'].pct_change(fill_method=None)
d['vol20'] = (d.groupby('sym', observed=True)['ret']
               .rolling(20, min_periods=15).std()
               .reset_index(level=0, drop=True) * np.sqrt(252))
```

## 19.2 Market data

**5. Fill every missing minute for every symbol: close carried forward, volume 0.**

```
full = pd.date_range('2026-10-19 09:15', '2026-10-19 15:29', freq='1min',
tz='Asia/Kolkata')
close = bars['close'].unstack('sym').reindex(full).ffill()        # wide:
minute x sym
volume = bars['volume'].unstack('sym').reindex(full).fillna(0)
filled = pd.concat({'close': close.stack(), 'volume': volume.stack()},
axis=1)   # back to long
```

Leading minutes before a symbol's first trade stay NaN, which is correct: there was no price yet.

**6. Detect sequence-number gaps per feed channel.**

```
feed = feed.sort_values(['channel', 'seq'])
gap = feed.groupby('channel')['seq'].diff()
missing = feed.loc[gap > 1, ['channel', 'seq']].assign(n_missing=gap[gap > 1]
- 1)
dups = feed[feed.duplicated(['channel', 'seq'], keep=False)]
```

**7. Classify trades against the prevailing quote and compute effective spread.**

```
tq = pd.merge_asof(trades.sort_values('ts'), quotes.sort_values('ts'),
on='ts', by='sym',
                   direction='backward', allow_exact_matches=False,
tolerance=pd.Timedelta('1s'))
tq['mid'] = (tq['bid'] + tq['ask']) / 2
tq['aggressor'] = np.select([tq['px'] >= tq['ask'], tq['px'] <= tq['bid']],
['buy', 'sell'], 'inside')
tq['eff_spread_bps'] = 2 * (tq['px'] - tq['mid']).abs() / tq['mid'] * 1e4
by_sym = tq.groupby('sym', observed=True).agg(
    pct_buy=('aggressor', lambda s: (s == 'buy').mean()),
    eff_spread_bps=('eff_spread_bps', 'median'))
```

**8. Time-weighted average spread per minute after removing duplicate quotes.**

```
q = (quotes.sort_values(['sym', 'ts'], kind='stable')
           .drop_duplicates(['sym', 'ts'], keep='last'))
q['spread'] = q['ask'] - q['bid']
next_ts = q.groupby('sym', observed=True)['ts'].shift(-1)        # next quote
of the same symbol
q['dur'] = (next_ts - q['ts']).dt.total_seconds()                # how long
this quote was live
q['w'] = q['spread'] * q['dur']
tw = q.set_index('ts').groupby('sym', observed=True)[['w',
'dur']].resample('1min').sum()
tw['tw_spread'] = tw['w'] / tw['dur']    # quotes spanning a minute boundary
count in their start minute
```

**9. Opening-range breakout: first close above the 09:15 to 09:29 high, per day.**

```
b = bars.between_time('09:15', '15:29')                    # one symbol, 1-
minute bars
day = pd.Series(b.index.date, index=b.index)
opening = b.between_time('09:15', '09:29')
orh = opening.groupby(opening.index.date)['high'].max()
b['orh'] = day.map(orh)
after = b.index.time >= pd.Timestamp('09:30').time()
break_up = after & (b['close'] > b['orh'])
first_break = break_up & (break_up.groupby(day).cumsum() == 1)
entries = b[first_break]
```

**10. Pairs trade: rolling hedge ratio, spread z-score and positions with hysteresis.**

```
la, lb = np.log(px['HDFCBANK']), np.log(px['ICICIBANK'])
beta = la.rolling(60).cov(lb) / lb.rolling(60).var()
spread = la - beta * lb
z = (spread - spread.rolling(60).mean()) / spread.rolling(60).std()

signal = pd.Series(np.select([z > 2, z < -2, z.abs() < 0.5], [-1.0, 1.0,
0.0], default=np.nan), index=z.index)
pos = signal.ffill().fillna(0)          # hold the position until an exit or
reversal condition fires
pnl = pos.shift(1) * (la.diff() - beta.shift(1) * lb.diff())   # trade on the
next bar
```

The `ffill` turns entry and exit conditions into a stateful position without a loop.

## 19.3 Research and data engineering

**11. Event study: average cumulative abnormal return from 5 days before to 5 days after each event.**

```
rows = []
for sym, d in events[['sym', 'date']].itertuples(index=False):   # few
events: a loop is fine
    i = resid.index.searchsorted(pd.Timestamp(d))
    if 5 <= i < len(resid) - 5:
        rows.append(resid[sym].iloc[i - 5:i + 6].to_numpy())
car = pd.DataFrame(rows, columns=range(-5, 6)).mean().cumsum()
```

### 12. Max drawdown, its peak date and trough date, for several strategies at once.

```
eq = (1 + rets).cumprod()                 # rets: date x strategy
dd = eq / eq.cummax() - 1
trough = dd.idxmin()
peak = pd.Series({c: eq[c].loc[:trough[c]].idxmax() for c in eq.columns})
result = pd.DataFrame({'max_dd': dd.min(), 'peak': peak, 'trough': trough})
```

### 13. Convert a folder of daily CSVs into a partitioned Parquet lake with a fixed schema.

```
from pathlib import Path
dtypes = {'sym': 'category', 'px': 'float64', 'qty': 'int64', 'seq': 'int64'}
for f in sorted(Path('raw').glob('*.csv.gz')):
    df = pd.read_csv(f, usecols=['ts_ns', *dtypes], dtype=dtypes)
    df['ts'] = pd.to_datetime(df.pop('ts_ns'), unit='ns', utc=True)
    df['date'] = df['ts'].dt.tz_convert('Asia/Kolkata').dt.strftime('%Y-%m-
%d')
    (df.sort_values(['sym', 'ts', 'seq'], kind='stable')
       .to_parquet('lake/trades', partition_cols=['date'],
compression='zstd', index=False))
```

### 14. Spot the bugs.

```
df = pd.read_csv('trades.csv')
df['ret'] = df['px'].pct_change()                       # (a)
df[df['qty'] > 100]['big'] = True                       # (b)
bars = df.set_index('ts')['px'].resample('1min').last() # (c)
sig = (bars > bars.rolling(20, center=True).mean())     # (d)
out = df.merge(ref, on='sym')                           # (e)
out['px'] = out['px'].fillna(method='bfill')            # (f)
```

Answers: (a) returns cross symbol boundaries and the file was never sorted by time; (b) chained assignment, never modifies `df`; (c) `ts` was read as text (no `parse_dates`), and the left-labelled bar contains the following 59 seconds; (d) a centred window uses future bars; (e) no `validate`, so duplicate keys in `ref` multiply rows, and `how` defaults to inner, silently dropping unmatched trades; (f) backward fill is look-ahead, and `fillna(method=...)` no longer exists in pandas 3.0.

**15. Point-in-time universe: keep only stocks that were in the index on each date.**

```
mem = membership.sort_values('effective_from')            # sym,
effective_from, in_index
panel = panel.sort_values('date')
panel = pd.merge_asof(panel, mem, left_on='date', right_on='effective_from',
by='sym', direction='backward')
universe_panel = panel[panel['in_index'].fillna(False).astype(bool)]
```

# 20. Pitfalls, cheat sheet and revision plan

Read the pitfalls and cheat sheet on the morning of the interview; use the plan for the days before.

## 20.1 Pitfalls checklist

### Selection and Copy-on-Write

- `df[mask]['col'] = x` and `df['col'][mask] = x` never modify `df` in pandas 3.0; use `df.loc[mask, 'col'] = x`.
- `Series[0]` is a label lookup, not "the first element"; use `.iloc[0]`.
- `loc` slices include the end label; `iloc` slices exclude it.
- A non-unique index makes `loc[label]` return a DataFrame.

### Alignment and the index

- Arithmetic aligns on labels: mismatched indexes produce NaN, not errors.
- `wide - row_series` aligns with columns; use `wide.sub(row_series, axis=0)`.
- Slicing an unsorted DatetimeIndex or MultiIndex fails or is slow; `sort_index()` once.

### dtypes

- One NaN turns an `int64` column into `float64`; use `Int64` for IDs and quantities with gaps.
- pandas 3.0 text columns are `str`, not `object`.
- pandas 3.0 parses timestamps to microseconds: `astype('int64')` gives microseconds unless you call `.dt.as_unit('ns')`.
- Ambiguous date strings need an explicit `format=`.

### Missing data

- `sum()` of an all-NaN group is 0; use `min_count=1`.
- `bfill`, `interpolate` and full-sample `fillna(mean)` are look-ahead.
- Forward fills must be grouped by symbol and session.

- `groupby` drops NaN keys by default; `merge` matches NaN keys to each other.

**GroupBy, merge and reshape**

- `pct_change` / `shift` / `diff` on a long table need `groupby('sym')` first.
- `first()` skips NaN; `nth(0)` does not.
- Merges without `validate=` can silently multiply rows; default `how='inner'` silently drops rows.
- `merge_asof` needs both sides sorted on the key and matching key dtypes.
- `pivot` fails on duplicate pairs; use `pivot_table` with an explicit `aggfunc`.

**Time series**

- Left-labelled bars contain data up to the end of the bar; shift or right-label them before joining to signals.
- Hourly bars need `offset='15min'` to start at 09:15.
- `rolling(center=True)` and `shift(-k)` features leak the future.
- `shift(1)` moves rows; `shift(1, freq=...)` moves timestamps.
- Naive and tz-aware timestamps cannot be compared or joined.

**I/O and formats**

- CSV loses dtypes, time zones and float precision; "NA" may be a real code.
- Parquet stores the index unless `index=False`; unsorted data defeats row-group statistics.
- Pickle is unsafe for untrusted files and fragile across versions.
- Many tiny Parquet files (one per symbol per day) are slow to list and read.

**Performance**

- `apply(axis=1)`, `iterrows` and Python lambdas in `groupby` are the usual bottlenecks.
- Inserting columns one by one fragments a frame; concatenating inside a loop is quadratic.
- `object` columns and unparsed dates make everything slower.

## 20.2 Cheat sheet

| Task | Code |
|---|---|
| Read only what you need | `pd.read_parquet(p, columns=[...], filters=[('date', '>=', d0)])` |
| Typed CSV read | `pd.read_csv(p, usecols=..., dtype=..., parse_dates=['ts'], date_format=...)` |

| Task | Code |
| --- | --- |
| Safe conditional write | `df.loc[mask, 'col'] = value` |
| Per-symbol returns | `df.groupby('sym')['px'].pct_change(fill_method=None)` |
| Group statistic back on rows | `df.groupby('date')['x'].transform('mean')` |
| Named aggregation | `g.agg(vol=('qty', 'sum'), hi=('px', 'max'))` |
| Cross-sectional z-score (wide) | `w.sub(w.mean(axis=1), axis=0).div(w.std(axis=1), axis=0)` |
| Cross-sectional rank (wide) | `w.rank(axis=1, pct=True)` |
| Long to wide / wide to long | `pivot(index=, columns=, values=)` / `melt(id_vars=)`; `unstack` / `stack` |
| OHLC bars | `s.resample('1min').ohlc()`; `groupby('sym')['px'].resample(...)` |
| Hourly bars from 09:15 | `resample('1h', offset='15min')` |
| Session filter | `df.between_time('09:15', '15:30')` |
| Trailing time window | `s.rolling('5min').sum()` |
| EMA | `s.ewm(span=n, adjust=False).mean()` |
| As-of join | `pd.merge_asof(l, r, on='ts', by='sym', direction='backward', tolerance=...)` |
| Checked merge | `merge(..., validate='many_to_one', indicator=True)` |
| Time zones | `tz_localize('UTC').tz_convert('Asia/Kolkata')` |
| Epoch ns to datetime | `pd.to_datetime(x, unit='ns', utc=True)` |
| Datetime to epoch ns | `s.dt.as_unit('ns').astype('int64')` |
| Business-day calendar | `CustomBusinessDay(holidays=hols)` |
| Memory check | `df.memory_usage(deep=True).sum()` |
| Partitioned Parquet write | `df.to_parquet(dir, partition_cols=['date'], compression='zstd', index=False)` |
| Fast local cache | `df.to_feather(p)` / `pd.read_feather(p)` |

| Task | Code |
|---|---|
| Hand off to Polars / DuckDB | `pl.from_pandas(df)` / `duckdb.sql('select ... from df').df()` |

## 20.3 Revision plan for the days before the interview

| Day | Read | Do |
|---|---|---|
| Day 1 | Sections 1 to 6 (objects, dtypes, selection, Index, missing data, transforms) | Predict every output in sections 1.2, 3.3 and 5.3 before running; write a typed `read_csv` and a cleaning chain from memory |
| Day 2 | Sections 7 to 11 (groupby, joins, reshaping, time series, windows) | Problems 1 to 7; build 1-minute and 09:15-aligned hourly bars and a trade-quote as-of join from scratch |
| Day 3 | Sections 12 to 17 (performance, I/O, file formats, other libraries, quant workflows, pandas 3.0) | Run the benchmark in 14.9 and note your own numbers; problems 8 to 13; rehearse the 14.10 storage answer |
| Day 4 | Section 18 aloud, section 19 timed, plus the NumPy guide's sections 18 to 20 | Problems 14 and 15 under time pressure; one 60-minute mock mixing NumPy, pandas and a storage-design question |
| Interview morning | Sections 20.1 and 20.2 | Light review only |

Prepare one story about a data pipeline from your own work, for example how the tick and option-chain data behind your expiry-day volatility research is stored, cleaned, aligned and checked for look-ahead. A concrete pipeline with the trade-offs you chose is the strongest answer to most storage and pandas design questions.